

Árvores B



- Nesta aula será apresentado o ADT árvore B que é uma árvore de grau m (m -vias) completa
- As árvores B são projetadas para funcionar bem em dispositivos de memória secundária (discos)

Introdução

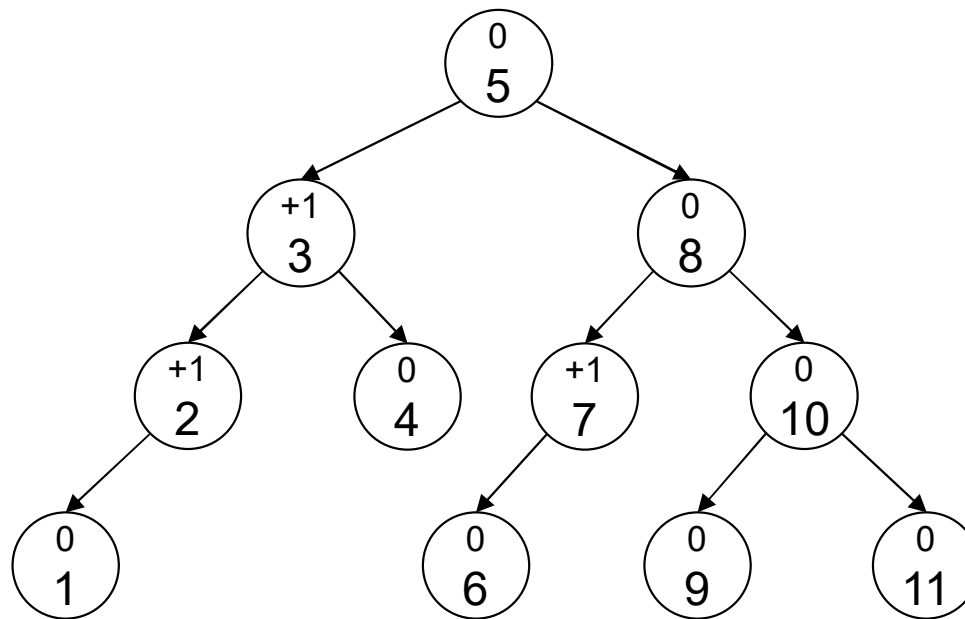
- ❑ Na pesquisa binária, as operações de busca em um conjunto com n elementos podem ser efetuadas em $O(\log_2 n)$, na realidade, $\lceil \log_2 n \rceil$ mesmo no pior caso
- ❑ No caso em que o conjunto de elementos se encontre em um arquivo em disco, sendo que os elementos (registros) estão ordenados por um campo chave então seria possível aplicar uma pesquisa binária para encontrar um elemento chave qualquer
 - Se os registros são recuperados do disco, um de cada vez, então a pesquisa binária exigiria um total de 20 acessos para um arquivo com um milhão (10^6) de entradas
- ❑ Entretanto, a inserção e remoção de elementos exigiria que o arquivo inteiro fosse reordenado
- ❑ Veremos uma técnica no qual inserções e remoções de registros tem apenas efeitos locais, sem exigir a reorganização total do arquivo, conhecida **indexação**

Introdução

- ❑ Um índice é uma estrutura de dados que permite rápido acesso aos dados com base em uma chave
- ❑ É importante que um índice, além de agilizar a busca, permita inserções e remoções de dados igualmente rápidas
- ❑ Por exemplo, nas árvores AVL as operações de busca, inserção e remoção em um conjunto com n elementos podem ser efetuadas em $O(\log_2 n)$, mesmo no pior caso
- ❑ Como as mesmas funções devem ser executadas para um índice, seria possível utilizar árvores AVL também para essa aplicação, ou seja, a árvore AVL residiria em disco
- ❑ Se os nós são recuperados do disco, um de cada vez, então a pesquisa de um índice com n entradas precisaria, no máximo, de $1.4 \cdot \log_2 n$ acessos (a profundidade máxima de uma árvore AVL é $1.4 \cdot \log_2 n$)

AVL como Índice em Disco

- ❑ No exemplo, se cada nó da AVL reside em disco (um registro representando cada nó), o acesso às chaves 1, 6, 9 e 11 requer a leitura de 4 registros



Representação (C++)

- Assim, um nó AVL em memória principal:

```
struct TreeNode
{ int key;           // chave
  int bal;           // -1, 0, +1
  TreeNode *leftNode, *rightNode; // subárvores
};
typedef TreeNode *TreePointer;
```

- Poderia ser alterado para ser representado em disco:

```
struct TreeNode
{ int key;           // chave
  int bal;           // -1, 0, +1
  int leftNode, rightNode; // subárvores
};
```

- Note que os campos **leftNode** e **rightNode** correspondem, nesta última representação, a número de registros (1, 2, ...) em disco

Representação (Pascal)

- Assim, um nó AVL em memória principal:

```
type TreePointer = ^TreeNode;
   TreeNode = record
       key : integer; // chave
       bal : -1..+1;  // -1, 0, +1
       leftNode, rightNode: TreePointer; // subárvores
   end;
```

- Poderia ser alterado para ser representado em disco:

```
type TreeNode = record
       key : integer; // chave
       bal : -1..+1;  // -1, 0, +1
       leftNode, rightNode : integer; // subárvores
   end;
```

- Note que os campos **leftNode** e **rightNode** correspondem, nesta última representação, a número de registros (1, 2, ...) em disco

Representação (Java)

- Assim, um nó AVL em memória principal:

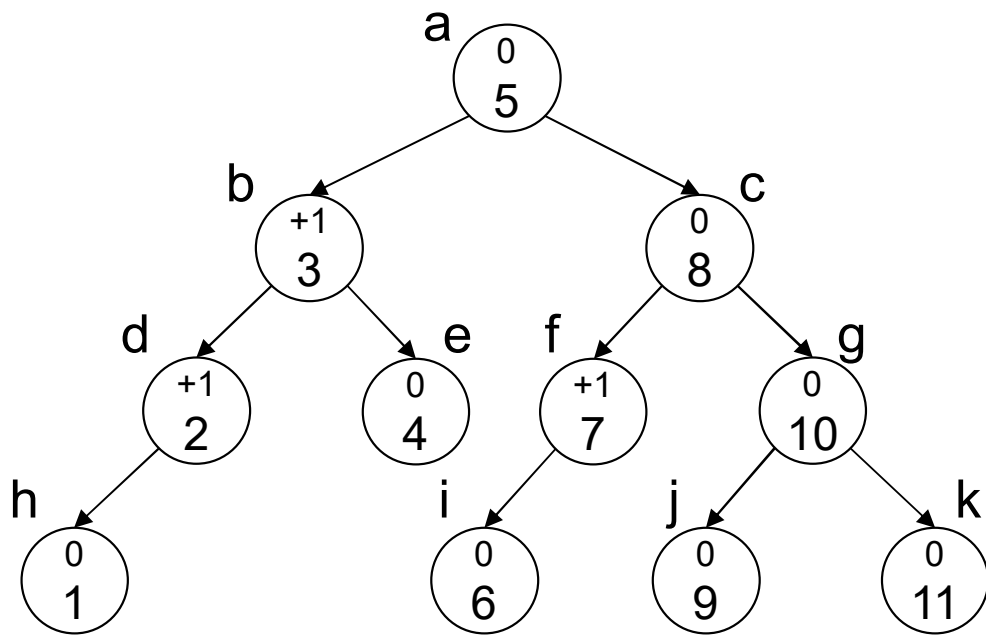
```
class TreeNode
{ int item;           // chave
  int bal;            // -1, 0, +1
  TreeNode leftNode, rightNode; // subárvores
}
```

- Poderia ser alterado para ser representado em disco:

```
class TreeNode
{ int item;           // chave
  int bal;            // -1, 0, +1
  int leftNode, rightNode; // subárvores
}
```

- Note que os campos **leftNode** e **rightNode** correspondem, nesta última representação, a número de registros (1, 2, ...) em disco

AVL como Índice em Disco



	key	bal	leftNode	rightNode
a	5	0	b	c
b	3	+1	d	e
c	8	0	f	g
d	2	+1	h	0
e	4	0	0	0
f	7	+1	i	0
g	10	0	j	k
h	1	0	0	0
i	6	0	0	0
j	9	0	0	0
k	11	0	0	0

AVL como Índice em Disco

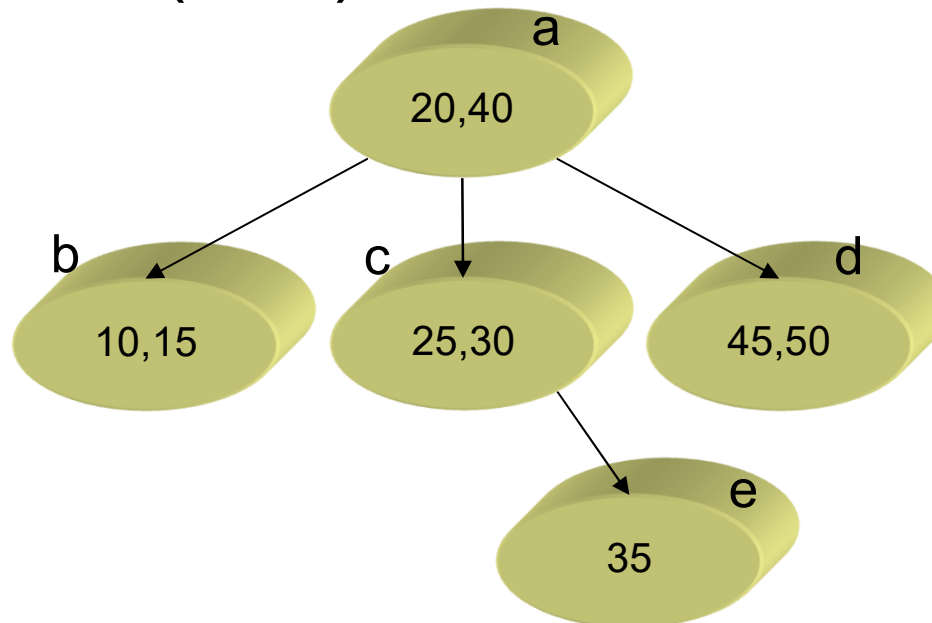
- ❑ No pior caso, isso significa cerca 28 acessos para um índice com um milhão (10^6) de entradas
- ❑ É possível reduzir consideravelmente o número de 28 acessos, usando uma árvore equilibrada, baseada numa árvore de busca em **m**-vias (árvores de busca de grau **m**), ao invés daquela baseada em uma árvore de busca binária balanceada

Histórico

- ❑ Em 1972 Bayer & McGreight publicaram o artigo *Organization and Maintenance of Large Ordered Indexes*
- ❑ Em 1979, o uso de árvores B era um padrão adotado em sistemas de arquivos de propósito geral para a manutenção de índices para bases de dados

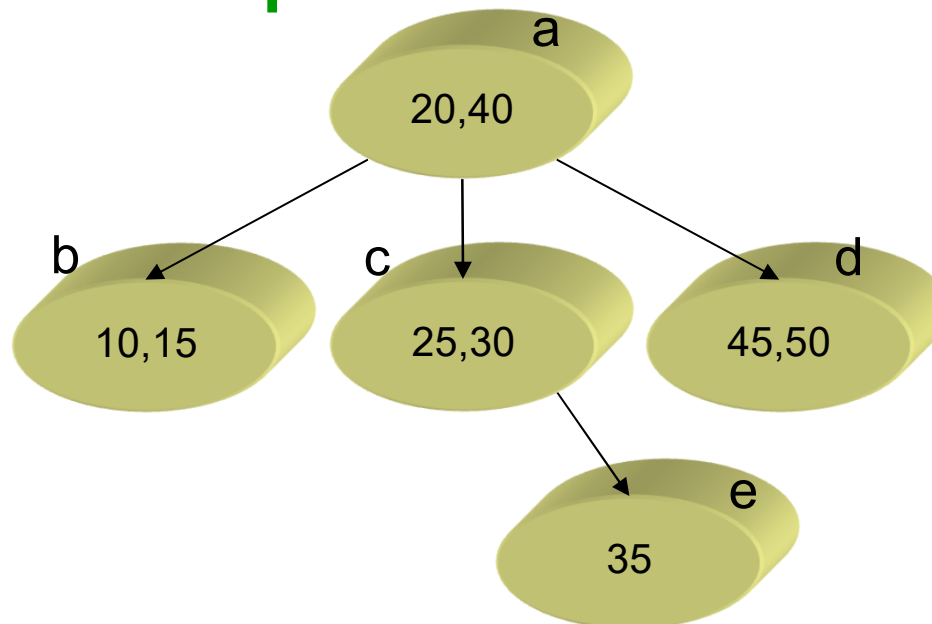
Árvore de Busca em m-Vias

- ❑ As árvores de busca em **m**-vias (**m**-árias ou de grau **m**) generalizam as árvores de busca binária de maneira natural
- ❑ Note que se um nó **p** da árvore tem **n** chaves então **p** tem **(n+1)** filhos



Árvore de Busca em m-Vias

- ❑ As chaves no nó **p** são usadas como pontos que separam o intervalo de chaves gerenciadas por **p** em $(n+1)$ intervalos, cada um gerenciado por um filho de **p**
- ❑ Em uma busca por uma valor, a decisão é feita de $(n+1)$ modos, com base em comparações com as **n** chaves armazenadas em **p**



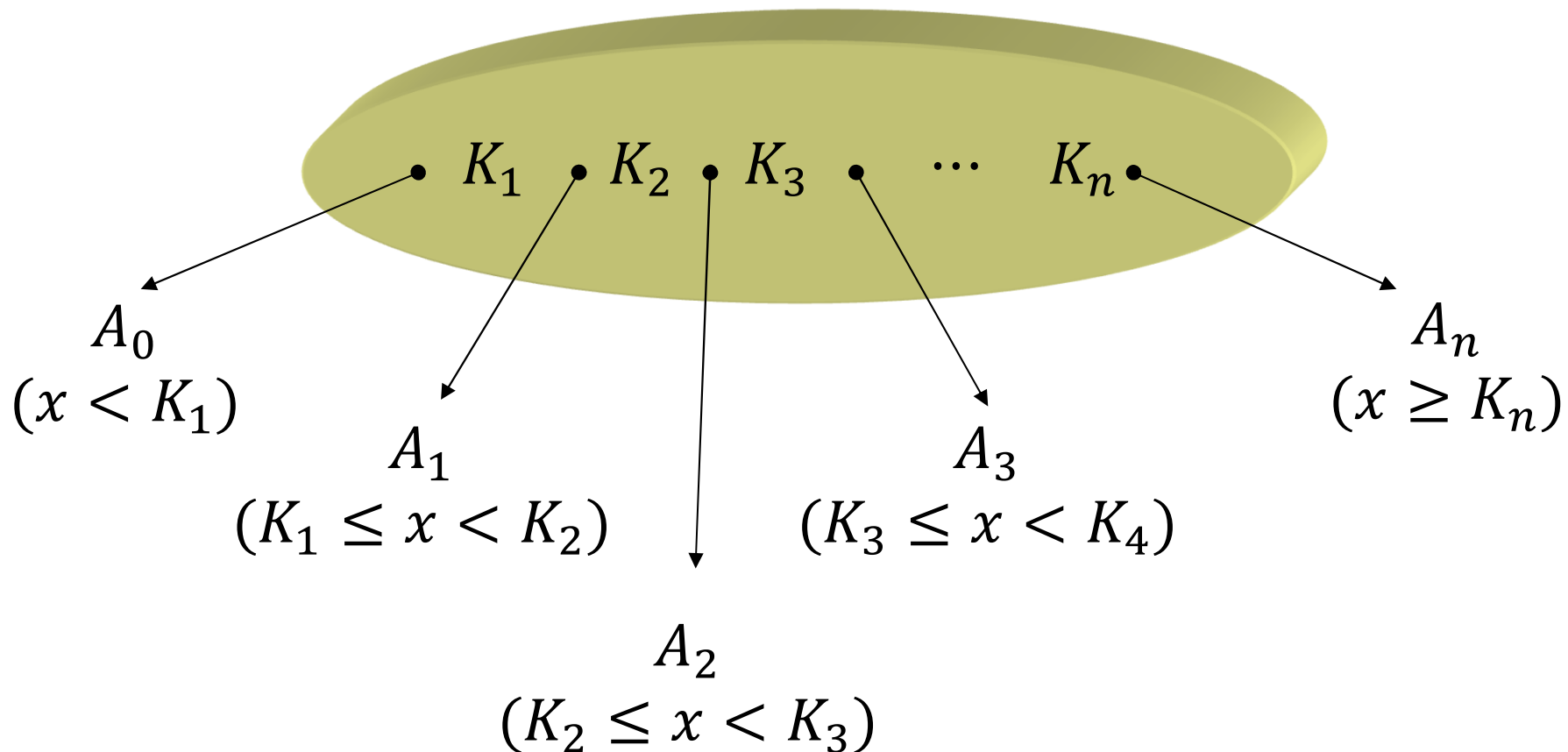
Árvore de Busca em m-Vias

- Uma árvore de busca de grau m (m -vias) é definida como:
 - Uma árvore vazia é uma árvore de busca de grau m
 - Uma árvore de busca T de grau m é uma árvore na qual todos os nós são de um grau menor ou igual a m
- Quando T for não vazia, ela tem as seguintes propriedades:
 - (a) T é um nó do tipo $n, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_n, A_n)$ onde
 - ❖ $A_i, 0 \leq i \leq n$ são ponteiros para as subárvores de T
 - ❖ $K_i, 1 \leq i \leq n$ são os valores de chave e $1 \leq n < m$
 - (b) $K_i < K_{i+1}, 1 \leq i < n$ (as chaves estão ordenada no nó)
 - (c) Todos os valores de chave na subárvore A_i são menores do que o valor da chave $K_{i+1}, 0 \leq i < n$
 - (d) Todos os valores de chave na subárvore A_n são maiores ou iguais a K_n
 - (e) As subárvores $A_i, 0 \leq i \leq n$, são também árvores de busca de grau m

Árvore de Busca em m-Vias

□ Nó: $n, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_n, A_n)$

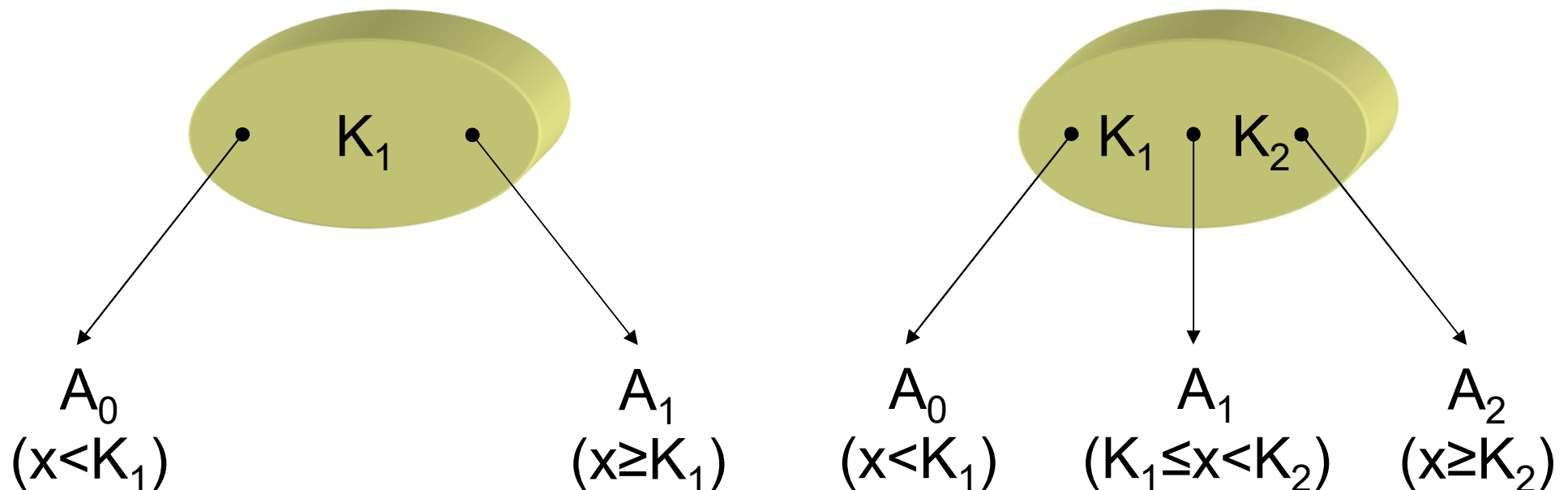
■ $K_1 < K_2 < \dots < K_n$



Árvore de Busca em 3-Vias

□ Para $m=3$, nós com uma ($n=1$) ou duas ($n=2$) chaves são admissíveis

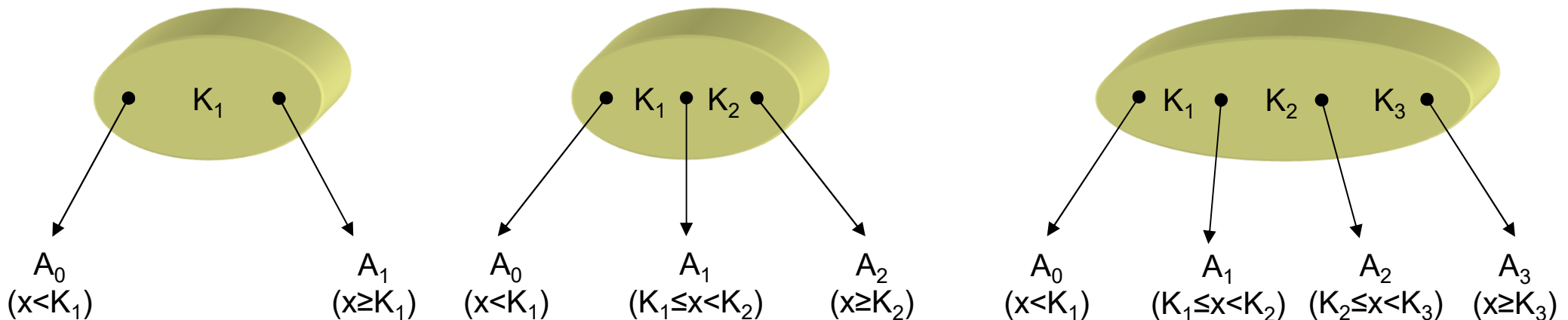
- $1, A_0, (K_1, A_1)$
- $2, A_0, (K_1, A_1), (K_2, A_2)$



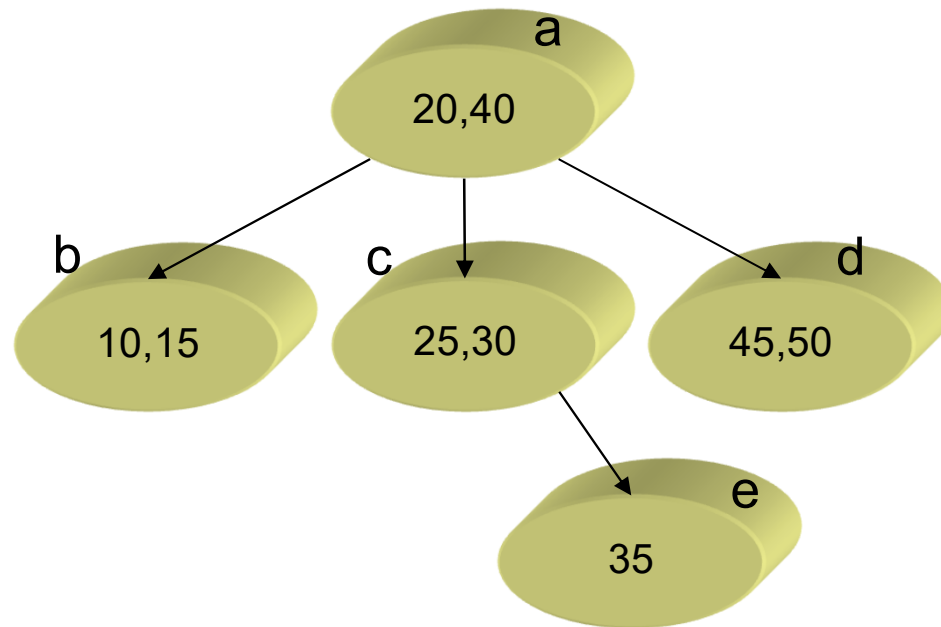
Árvore de Busca em 4-Vias

□ Para $m=4$, nós com $n=1, 2$ ou 3 chaves são admissíveis

- $1, A_0, (K_1, A_1)$
- $2, A_0, (K_1, A_1), (K_2, A_2)$
- $3, A_0, (K_1, A_1), (K_2, A_2), (K_3, A_3)$



Exemplo AB em 3-vias



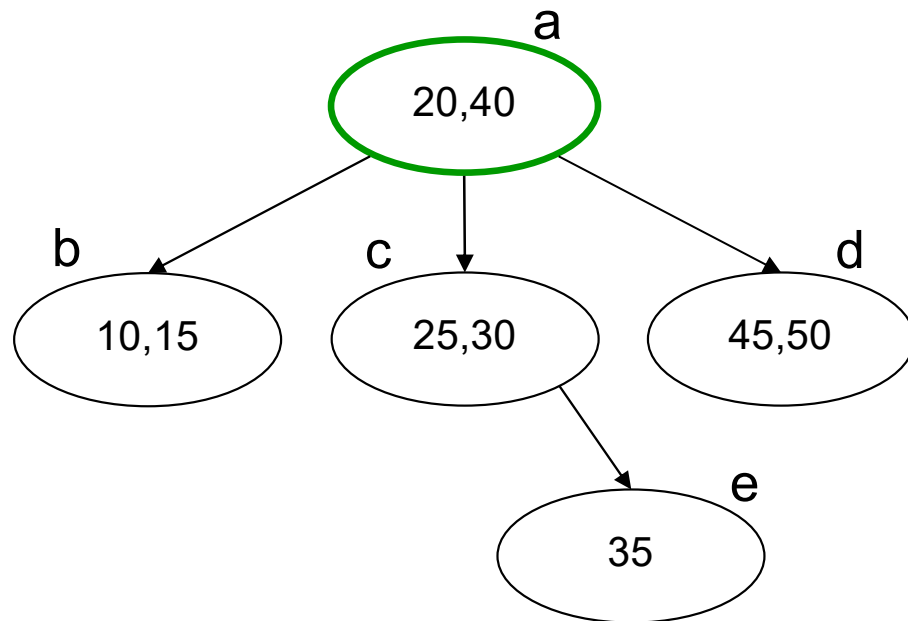
$n, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_n, A_n)$

Nó	Formato esquemático
a	2,b,(20,c),(40,d)
b	2,0,(10,0),(15,0)
c	2,0,(25,0),(30,e)
d	2,0,(45,0),(50,0)
e	1,0,(35,0)

Busca de um elemento

- ❑ Para buscar um valor x na árvore T inicia-se pela raiz, determinando o valor de i para o qual $K_i \leq x < K_{i+1}$
 - por conveniência define-se $K_0 = -\infty$ e $K_{n+1} = +\infty$, onde $-\infty$ é menor do que todos os valores de chave permitidas e $+\infty$ é maior do que todos os valores de chave permitidas
- ❑ Se T é vazia então a busca falhou (x não se encontra em T)
- ❑ Se $x = K_i$ então a busca teve sucesso
- ❑ Se $x \neq K_i$, pela definição da árvore, então x só poderá estar na subárvore A_i (quando x se encontra na árvore), ou seja, a busca deve continuar a partir do nó A_i
- ❑ Quando n (o número de chaves) for “grande”, a pesquisa do valor apropriado de i deve ser conduzida usando busca binária; para n “pequeno” uma busca sequencial é mais adequada

Busca de $x=35$



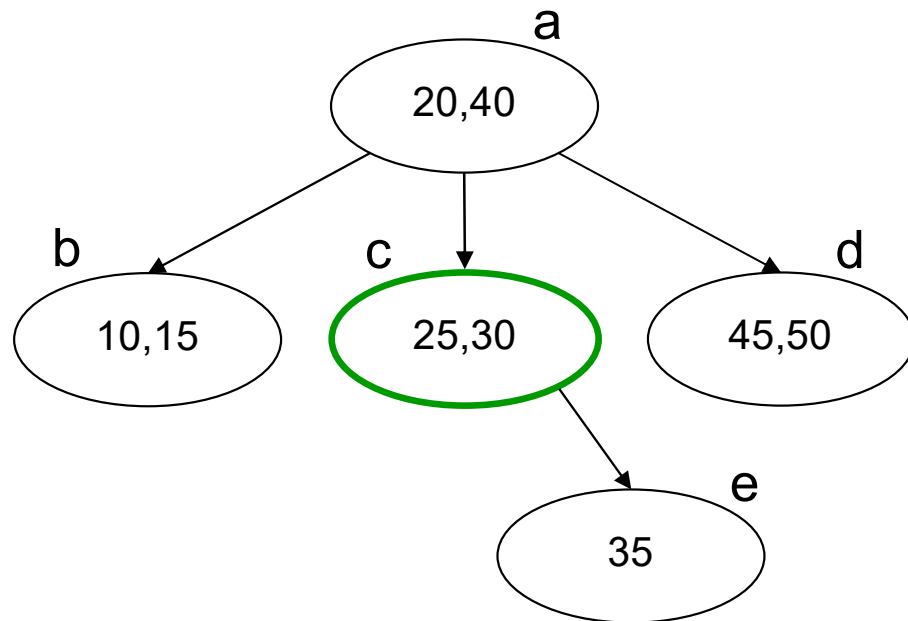
A busca inicia pela raiz (nó **a**),
determinando i tal que $K_i \leq x < K_{i+1}$.
Como $i=1$ (e $x \neq K_1$) então a busca
deve continuar na subárvore A_1 (nó **c**)

$n, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_n, A_n)$

Nó	Formato esquemático
a	2,b,(20,c),(40,d)
b	2,0,(10,0),(15,0)
c	2,0,(25,0),(30,e)
d	2,0,(45,0),(50,0)
e	1,0,(35,0)

	K_0	K_1	K_2	K_3
a	$[-\infty]$	20	40	$[\infty]$

Busca de $x=35$



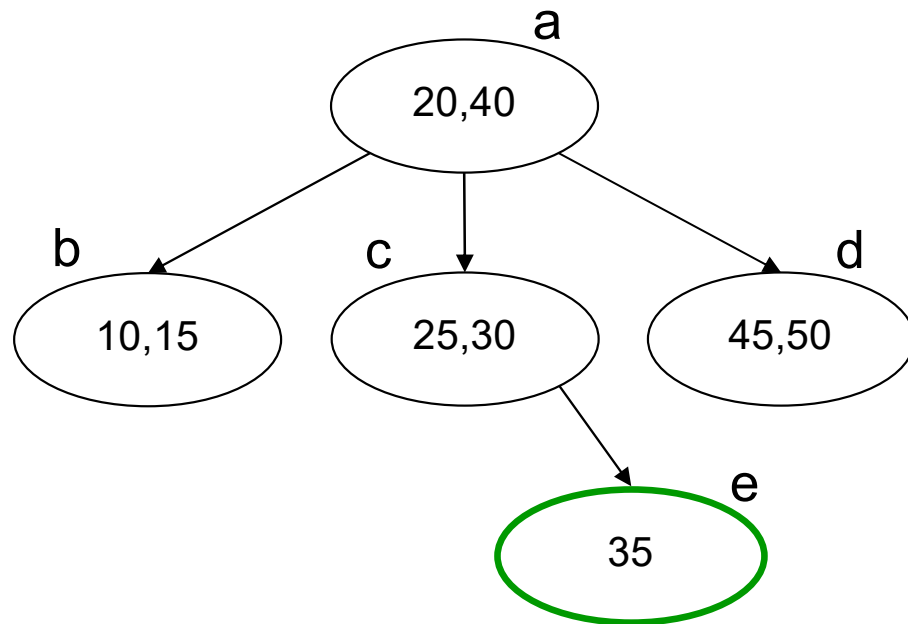
A busca continua pelo nó **c**,
determinando i tal que $K_i \leq x < K_{i+1}$.
Como $i=2$ (e $x \neq K_2$) então a busca
deve continuar na subárvore A_2 (nó **e**)

$n, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_n, A_n)$

Nó	Formato esquemático
a	2,b,(20,c),(40,d)
b	2,0,(10,0),(15,0)
c	2,0,(25,0),(30,e)
d	2,0,(45,0),(50,0)
e	1,0,(35,0)

	K_0	K_1	K_2	K_3
c	$[-\infty]$	25	30	$[\infty]$

Busca de $x=35$



A busca continua pelo nó **e**,
determinando i tal que $K_i \leq x < K_{i+1}$.
Como $i=1$ (e $x=K_1$) então a busca
termina com sucesso

$n, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_n, A_n)$

Nó	Formato esquemático
a	2,b,(20,c),(40,d)
b	2,0,(10,0),(15,0)
c	2,0,(25,0),(30,e)
d	2,0,(45,0),(50,0)
e	1,0,(35,0)

	K_0	K_1	K_2
e	$[-\infty]$	35	$[\infty]$

Algoritmo de Busca

```
// pré: procure na árvore de busca de m-vias T residente em disco
// pela chave x. O formato do nó é  $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$ ,  $n < m$ .
// pós: retorna triplo  $(p, i, j)$ ;  $j = \text{true}$  significa que x foi
// encontrado no nó p, chave  $K_i$ . Caso contrário  $j = \text{false}$  e p é o
// nó onde x pode ser inserido
```

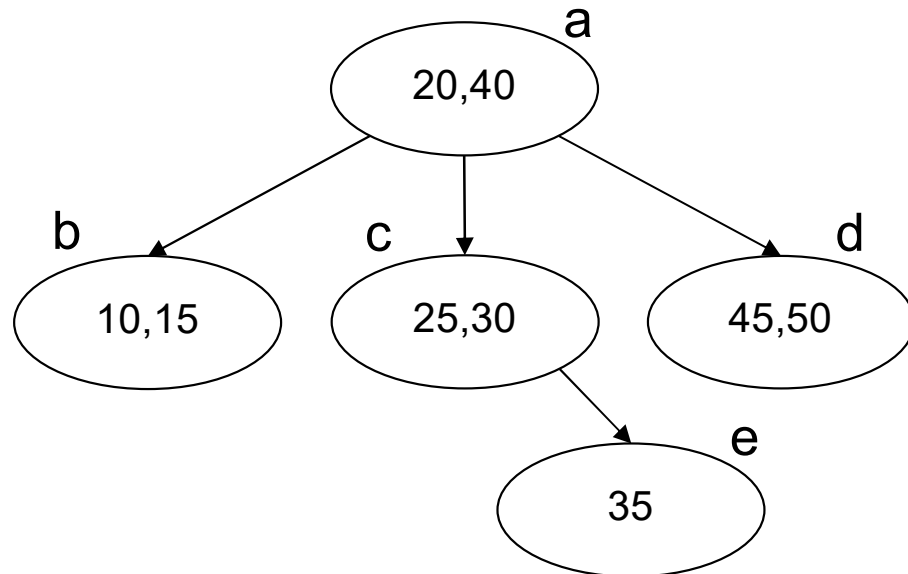
```
function mSearch(T, x)
```

```
  1  p ← T; q ← 0;                                // q é o pai de p
  2  while p ≠ 0 do
  3    leia nó p do disco;
  4    seja p da forma:  $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$ 
  5     $K_0 \leftarrow -\infty$ ;  $K_{n+1} \leftarrow +\infty$ ;
  6    Encontre i tal que  $K_i \leq x < K_{i+1}$ ;
  7    if  $x = K_i$  then return (p, i, true); endif // x foi encontrado
  8    q ← p; p ←  $A_i$ ;
  9  endwhile
    // x não está em T; retorne nó onde pode ser feita a inserção
10  return (q, i, false);
end mSearch
```

Pontos Importantes

- ❑ Na prática, quando a árvore de busca representa um índice, os duplos (K_i, A_i) em cada nó devem ser acrescidos na forma de triplos (K_i, A_i, B_i) , onde B_i é o endereço do registro com a chave K_i no arquivo principal
- ❑ No caso de discos
 - Esse endereço será formado pelos números de cilindro, superfície, trilha e setor que deve ser acessado para recuperar o registro
 - Os A_i 's, $0 \leq i \leq n$ são os números de endereços do nó raiz das subárvores; Como esses nós encontram-se também no disco, os A_i 's são números de cilindro, superfície, trilha e setor
- ❑ No caso de arquivos de acesso aleatório
 - Esse endereço será formado pelos números de registros, cujos valores permitidos dependem da linguagem de programação utilizada.
 - ❖ Em C/C++/Pascal os registros são numerados a partir de zero
 - Os A_i 's, $0 \leq i \leq n$ são números de registros

Exemplo



$n, A_0, (K_1, A_1, B_1), (K_2, A_2, B_2), \dots, (K_n, A_n, B_n)$
Arquivo Índice

Registro	Formato esquemático
a	2,b,(20,c,#2),(40,d,#1)
b	2,0,(10,0,#3),(15,0,#5)
c	2,0,(25,0,#9),(30,e,#8)
d	2,0,(45,0,#7),(50,0,#6)
e	1,0,(35,0,#4)

Arquivo Principal

Registro	Chave	Demais campos
#1	40	
#2	20	
#3	10	
#4	35	
#5	15	
#6	50	
#7	45	
#8	30	
#9	25	

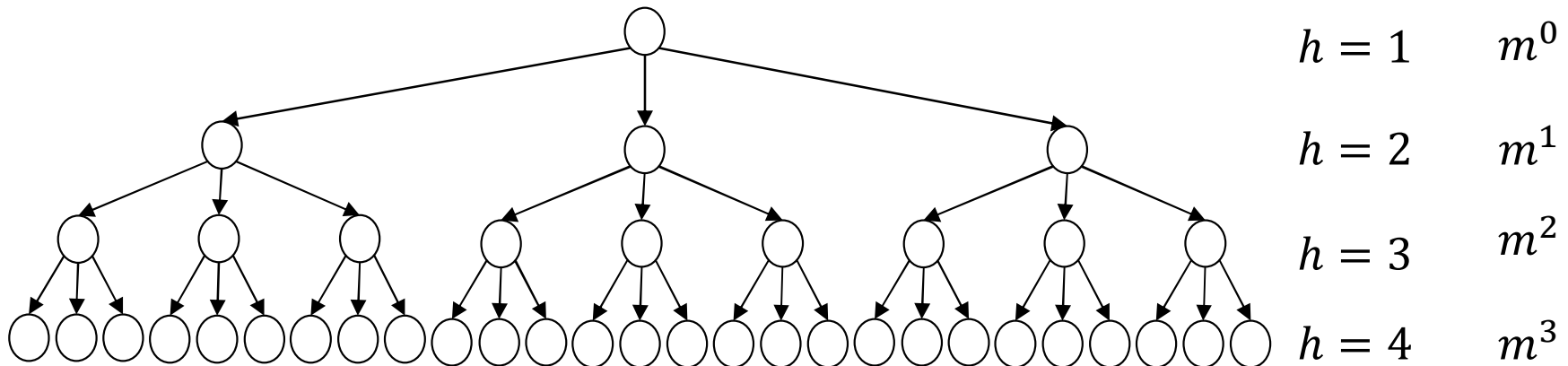
Análise de mSearch

- ❑ O número máximo de acessos ao disco é igual à altura da árvore T
- ❑ A meta é minimizar a quantidade de acessos, necessários para conduzir uma busca, porque os acessos individuais ao disco são bastante dispendiosos em relação ao tempo necessário para processar um nó (para determinar o próximo nó a ser acessado, linhas 4-8); isso equivale a minimização da altura de árvore de busca

Número Máximo de Nós

- Em uma árvore m -vias e altura $h \geq 1$ (considerando a raiz no nível 1), o **número máximo de nós** (estruturas de armazenamento) S_{max} é

$$S_{max}(m, h) = m^0 + m^1 + \dots + m^{h-1} = \sum_{i=0}^{h-1} m^i = \frac{m^h - 1}{m - 1}$$

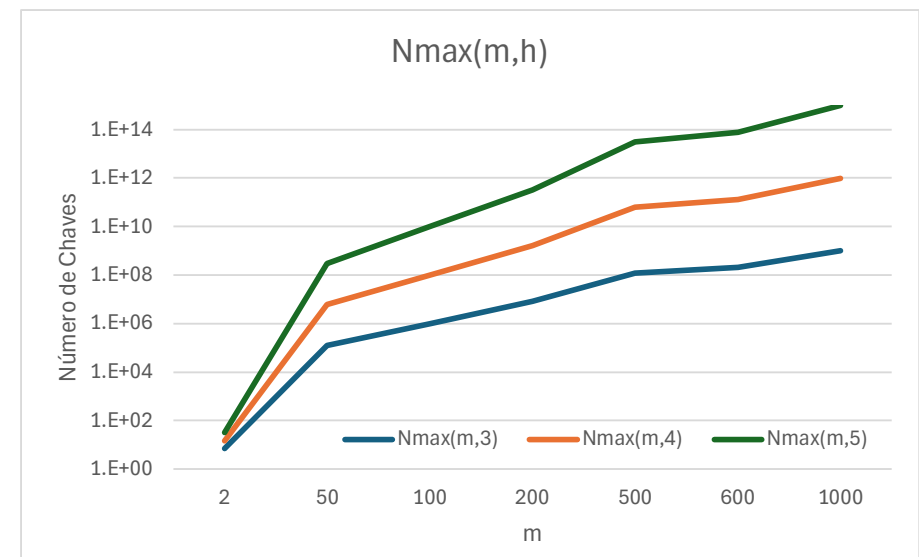


Número Máximo de Chaves

- Como cada nó tem, no máximo, $(m - 1)$ chaves, então o **número máximo de chaves** N_{max} é

$$N_{max}(m, h) = S_{max}(m, h) \cdot (m - 1) = \frac{m^h - 1}{m - 1} (m - 1) = m^h - 1$$

m	$N_{max}(m, 3)$	$N_{max}(m, 4)$	$N_{max}(m, 5)$
2	8 - 1	16 - 1	32 - 1
50	125000 - 1	6250000 - 1	312500000 - 1
100	1000000 - 1	100000000 - 1	10000000000 - 1
200	8000000 - 1	1600000000 - 1	320000000000 - 1
500	125000000 - 1	62500000000 - 1	31250000000000 - 1
600	216000000 - 1	129600000000 - 1	77760000000000 - 1
1000	1000000000 - 1	1000000000000 - 1	1000000000000000 - 1



Altura Mínima

- A altura mínima é obtida quando a árvore de busca máxima o número de nós N , ou seja, $N_{max}(m, h) \geq N$

$$m^{h_{min}} - 1 \geq N$$

$$m^{h_{min}} \geq N + 1$$

$$h_{min} \geq \log_m(N + 1)$$

- Portanto,

$$h_{min}(m, N) = \lceil \log_m(N + 1) \rceil$$

Árvores B

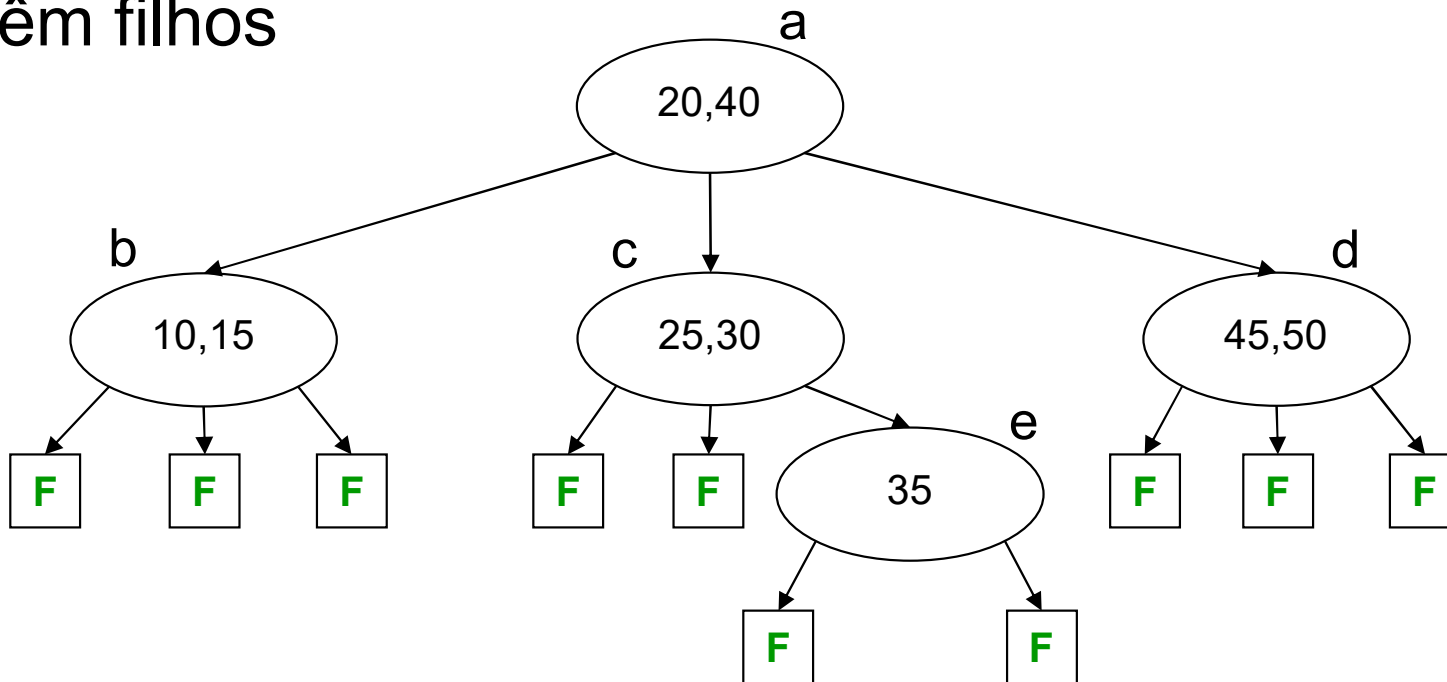
- ❑ Dessa forma, o potencial das árvores de busca de ordem alta é muito maior do que daquelas de ordem baixa
- ❑ Para atingir um desempenho próximo das melhores árvores de busca em **m**-vias, para um determinado número de entradas **N**, é necessário que a árvore de busca seja equilibrada
- ❑ A variedade particular das árvores de busca em **m**-vias, equilibradas, que vamos considerar aqui, é conhecida como **árvores B**

Nós de Falha

- ❑ Para definir uma árvore B será conveniente introduzir o conceito de nós de falha
- ❑ Um **nó de falha** representa o nó que pode apenas ser alcançado, durante a pesquisa, se o valor X não se encontra na árvore
- ❑ Cada subárvore vazia ($\text{raiz}=0$) é o ponto que pode ser alcançado durante a pesquisa, apenas se X não está na árvore
- ❑ Essas subárvores vazias serão substituídas, por conveniência, por **nós hipotéticos** chamados de **nós de falha**
- ❑ Esses nós serão desenhados como quadrados e marcados com um '**F**'

Nós de Falha

- ❑ Ressalta-se que a estrutura da árvore não contém efetivamente nenhum nó de falha, mas apenas o valor zero (ou outro valor indicando subárvore vazia) quando ele ocorrer
- ❑ É fácil verificar que os nós de falha são os únicos que não têm filhos



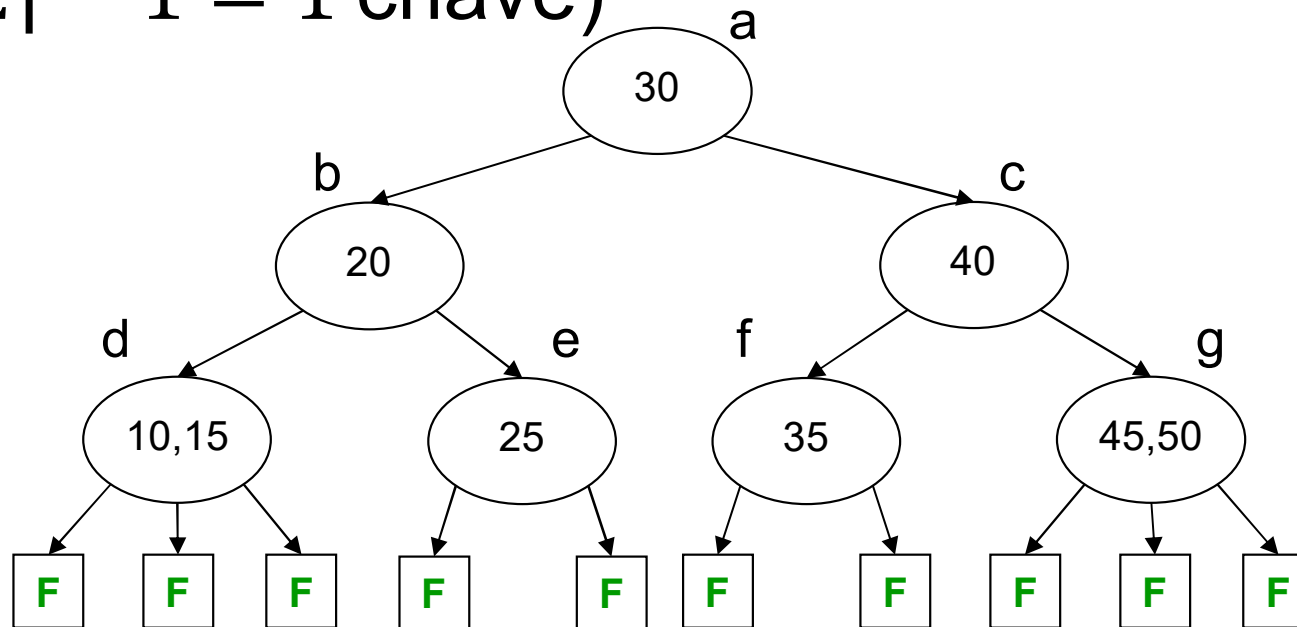
Árvores B

- ❑ Uma árvore T do tipo B e ordem m é uma árvore de busca em m -vias, que é ou vazia ou de altura $h \geq 1$ e satisfaz as seguintes propriedades:
 - (a) a raiz tem pelo menos 2 filhos
 - (b) todos os nós, exceto a raiz e nós de falha têm, no mínimo, $\lceil m/2 \rceil$ filhos (ou seja, $\lceil m/2 \rceil - 1$ chaves)
 - (c) todos os nós de falha estão no mesmo nível
- ❑ $\lceil x \rceil$ é o teto de x , ou seja, é o menor inteiro $\geq x$
- ❑ A raiz da árvore encontra-se no nível $h = 1$

m	No. Mínimo de filhos	No. Mínimo de chaves
3	2	1
4	2	1
5	3	2
10	5	4
50	25	24
100	50	49
200	100	99
600	300	299
1000	500	499

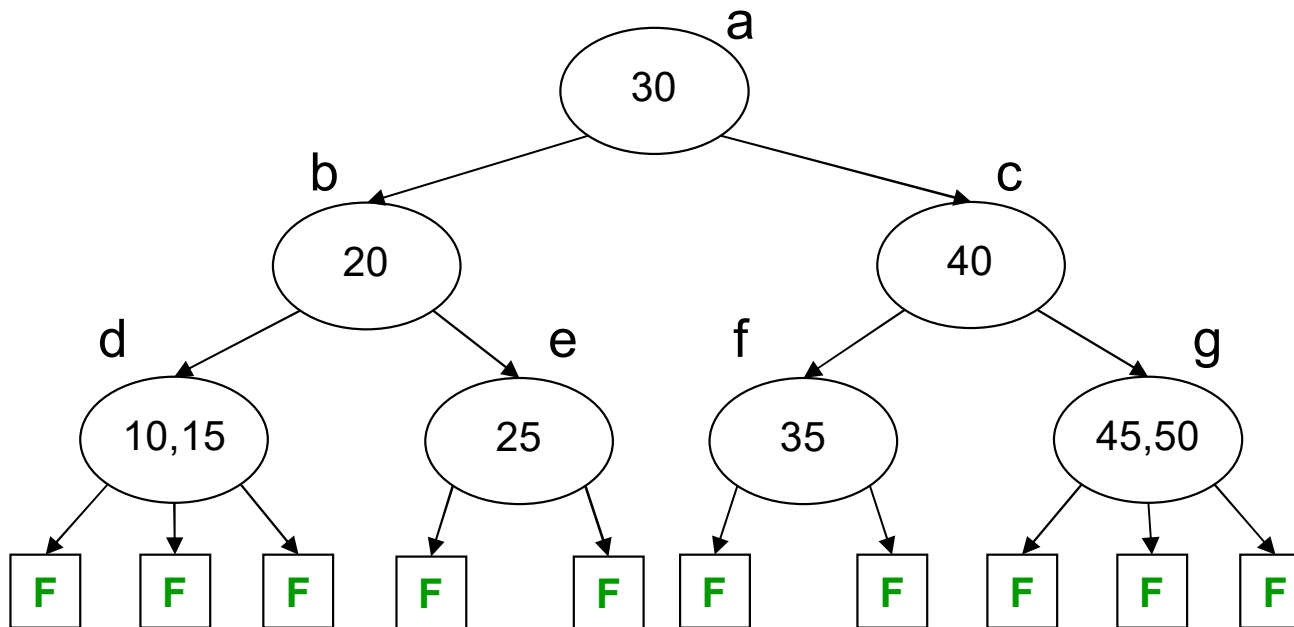
Árvores B

- ❑ A árvore da figura é uma árvore B de ordem 3
- ❑ Observe que todos os nós, exceto a raiz e nós de falha, têm pelo menos $\lceil 3/2 \rceil = 2$ filhos (ou $\lceil 3/2 \rceil - 1 = 1$ chave)



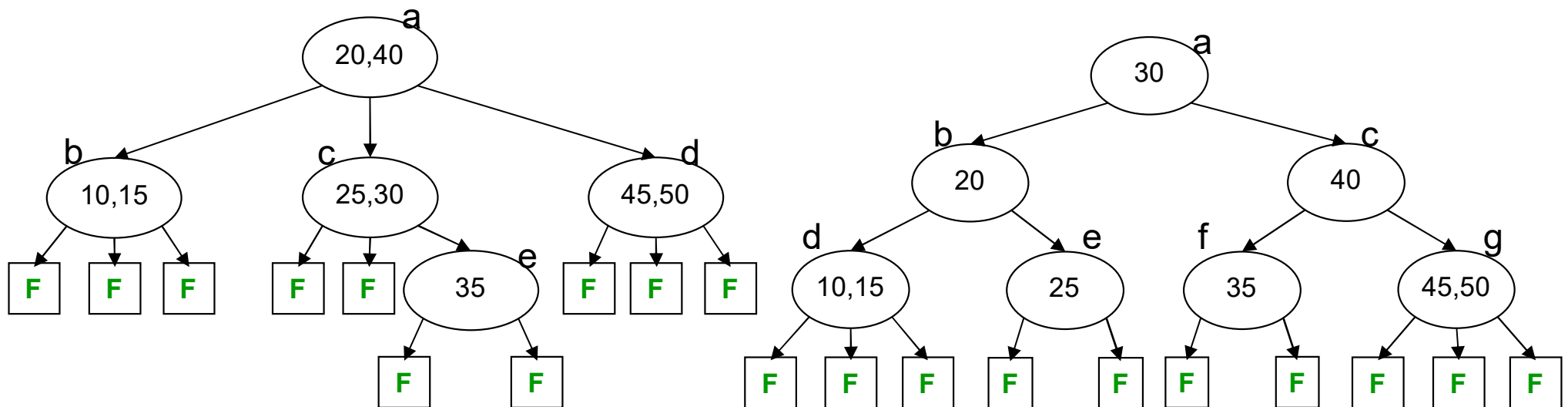
Árvores B

- ❑ Note que todos os nós, exceto os de falha, são de grau 2 ou 3
- ❑ De fato, a definição de uma árvore B de ordem 3 implica que todos os nós, exceto os de falha, devem ter grau 2 ou 3
- ❑ Por esse motivo, as árvores B de ordem 3 são também conhecidas como árvores 2-3



Árvores B

- Enquanto o número total de nós não-de-falha pode ser maior em uma árvore B, do que o número de tais nós na melhor árvore de busca em **m**-vias de mesma ordem (a árvore da esquerda tem apenas 5 nós, enquanto a árvore da direita 7) constataremos que é mais fácil inserir e eliminar nós na árvore B do que manter em qualquer tempo, a melhor possível árvore de busca em **m**-vias



Árvores B

- As razões de usarmos para índices as árvores B, ao invés de árvores de busca em **m**-vias otimizadas, são as mesmas daqueles que determinaram o uso de árvores AVL para manter as tabelas internas dinâmicas e não as árvores de busca binária otimizadas

Número Mínimo de Nós

- A quantidade mínima $S_{min}(m, h)$ de nós em T pode ser determinada usando a definição de uma árvore do tipo B, que no caso de $h > 1$
- Um nó que é a raiz (para $h = 1$ já que $h > 1$)
 - A raiz tem pelo menos 2 filhos (no mínimo ela tem 1 chave) e, portanto, há pelo menos dois nós no nível $h = 2$
 - Cada um desses nós deve ter pelo menos $\lceil m/2 \rceil$ filhos, portanto há, pelo menos, $2\lceil m/2 \rceil$ nós no nível $h = 3$
 - O nível $h = 4$ deve ter, pelo menos, $2\lceil m/2 \rceil^2$ nós
 - Continuando este raciocínio, haverá, pelo menos, $2\lceil m/2 \rceil^{h-2}$ nós no nível h
 - Todos esses nós são não-de-falha

$$S_{min}(m, h) = 1 + 2 \sum_{i=2}^h (\lceil m/2 \rceil^{i-2}) = 1 + 2 \left(\frac{\lceil m/2 \rceil^{h-1} - 1}{\lceil m/2 \rceil - 1} \right)$$

Número Mínimo de Chaves

- Se os valores de chave na árvore são $K_1, K_2, \dots, K_{N_{min}}$ e $K_i < K_{i+1}, 1 \leq i < N_{min}$ então existem $(N_{min}+1)$ nós de falha
- Isso é assim, porque os nós de falhas acontecem para X tal que $K_i < X < K_{i+1}, 0 \leq i \leq N_{min}$ e $K_0 = -\infty, K_{N_{min}+1} = \infty$
- Ou seja, pode-se alcançar $(N_{min}+1)$ nós de falha distintos, buscando um valor de X que não está na T
- Por isso, temos
 - $N_{min} + 1 =$ quantidade de nós de falha em T
 - $N_{min} + 1 =$ quantidade de nós no nível $h + 1$
 - $N_{min} + 1 \geq 2\lceil m/2 \rceil^{h-1}$
 - e, desse modo, $N_{min}(m, h) \geq 2\lceil m/2 \rceil^{h-1} - 1$ para $h > 1$

Número Mínimo de Chaves

- Uma forma alternativa de encontrar a ocupação mínima de chaves seria considerar que a raiz ($h = 1$) precisa ter 2 filhos (1 chave)
- Os demais nós devem ter, pelo menos, $\lceil m/2 \rceil$ filhos e, portanto, $\lceil m/2 \rceil - 1$ chaves
 - No nível $h > 1$ há $2\lceil m/2 \rceil^{h-2}$ nós

$$N_{min}(m, h) \geq 1 + 2 \sum_{i=2}^h (\lceil m/2 \rceil - 1) \cdot \lceil m/2 \rceil^{i-2}$$

$$N_{min}(m, h) \geq 1 + 2(\lceil m/2 \rceil - 1) \sum_{i=2}^h \lceil m/2 \rceil^{i-2}$$

$$N_{min}(m, h) \geq 2\lceil m/2 \rceil^{h-1} - 1$$

Altura Máxima

- A altura máxima ocorre todos os nós possuem o preenchimento mínimo permitido: a raiz com 1 chave (2 filhos) e os demais nós com $\lceil m/2 \rceil - 1$ chaves ($\lceil m/2 \rceil$ filhos)
- Como visto, o número mínimo de chaves N para uma altura h é:
 - $N \geq 2\lceil m/2 \rceil^{h-1} - 1 \Rightarrow N + 1 \geq 2\lceil m/2 \rceil^{h-1}$
 - $\frac{N+1}{2} \geq \lceil m/2 \rceil^{h-1} \Rightarrow \log_{\lceil m/2 \rceil} \left(\frac{N+1}{2} \right) \geq h - 1$
 - $\log_{\lceil m/2 \rceil} \left(\frac{N+1}{2} \right) + 1 \geq h$
- Portanto $h_{max}(m, N) = \left\lfloor \log_{\lceil m/2 \rceil} \left(\frac{N+1}{2} \right) \right\rfloor + 1$

Número de Chaves

- ❑ Recorde que número máximo de acessos que precisam ser feitos para uma busca é h
- ❑ Usando uma árvore B de ordem $m = 200$, temos as alturas máximas para os valores de N na tabela ao lado
- ❑ Por essa razão, a utilização de árvores B de ordem alta, resulta em um índice de árvore que pode ser pesquisado, minimizando o número de acessos ao disco, mesmo para uma quantidade muito grande de chaves

N	$h_{max}(200, N)$
$2 \times 10^3 - 2$	2
$2 \times 10^4 - 2$	2
$2 \times 10^5 - 2$	3
$2 \times 10^6 - 2$	3
$2 \times 10^7 - 2$	4
$2 \times 10^8 - 2$	4
$2 \times 10^9 - 2$	5
$2 \times 10^{10} - 2$	5

Número de Chaves/Altura

□ Números mínimo e máximo de chaves para árvores B de ordem m e altura $h \geq 1$

- $N_{min}(m, h) = 2\lceil m/2 \rceil^{h-1} - 1$
- $N_{max}(m, h) = m^h - 1$

m=3		
h	Nmin	Nmax
1	1	2
2	3	8
3	7	26
4	15	80
5	31	242
6	63	728
7	127	2186
8	255	6560
9	511	19682
10	1023	59048

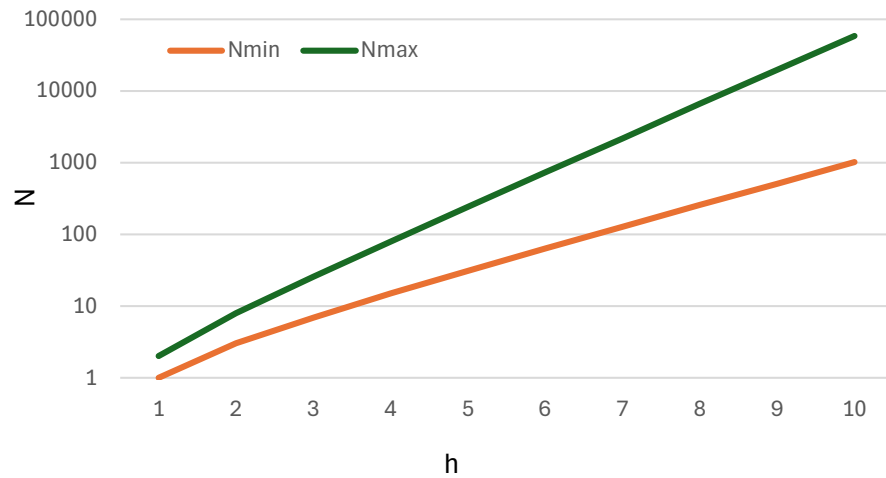
□ Alturas mínima e máxima para árvores B de ordem m contendo N chaves

- $h_{min}(m, N) = \lceil \log_m(N + 1) \rceil$
- $h_{max}(m, N) = \left\lceil \log_{\lceil m/2 \rceil} \left(\frac{N+1}{2} \right) \right\rceil + 1$

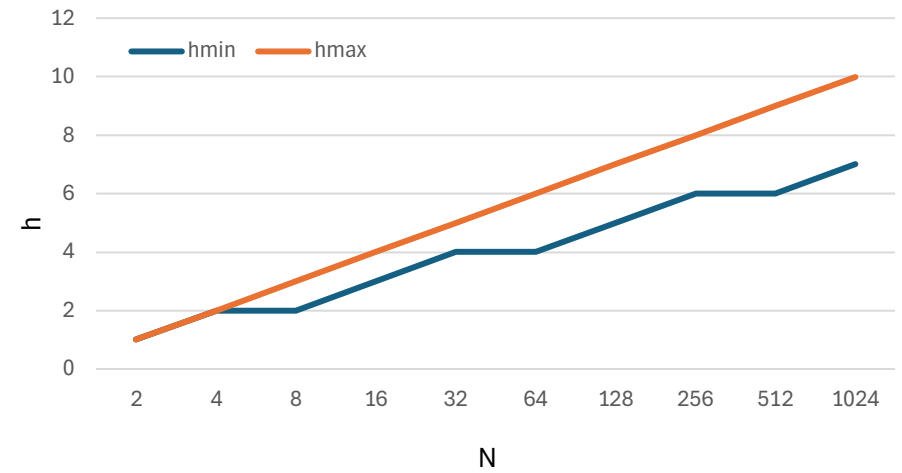
m=3		
N	hmin	hmax
1	1	1
2	1	1
3	2	2
4	2	2
5	2	2
6	2	2
7	2	3
8	2	3
9	3	3
10	3	3

Número de Chaves/Altura

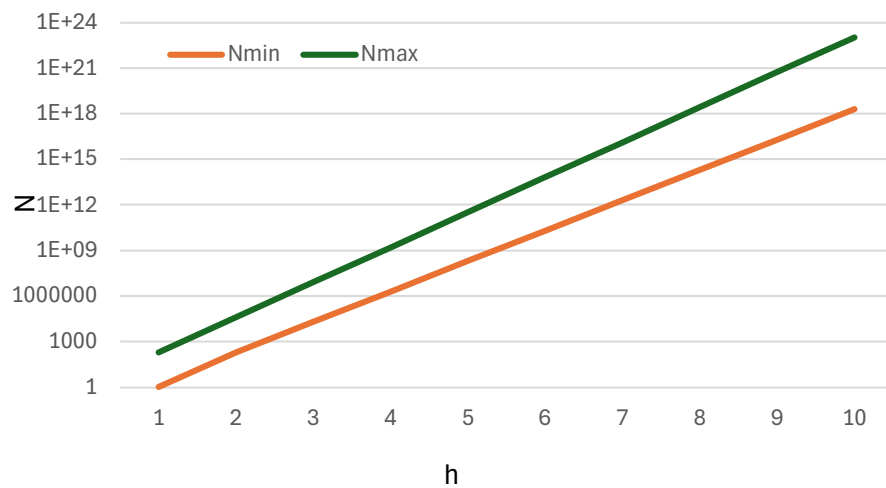
Número de chaves (m=3)



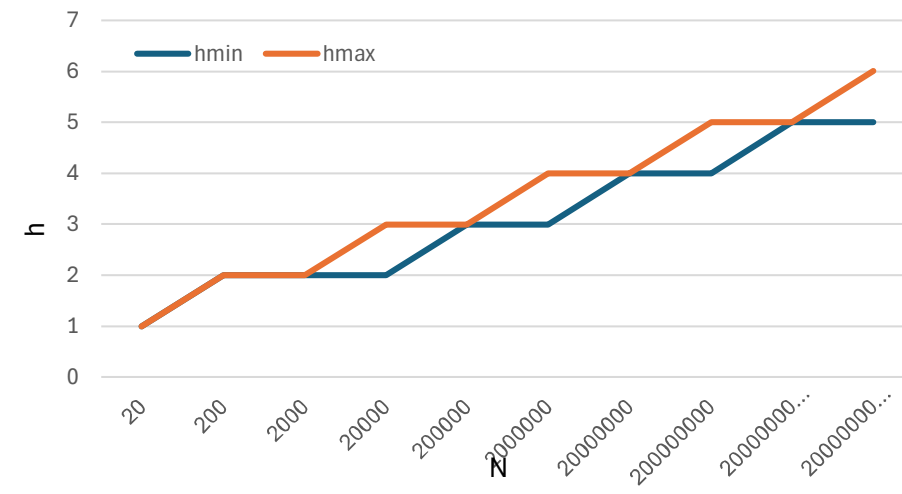
Altura (m=3)



Número de chaves (m=200)



Altura (m=200)



O custo de ter uma árvore com apenas 50% de ocupação...

□ Sabemos que $\log_b a = \frac{\log_c a}{\log_c b} = \frac{\log_m a}{\log_m b} = \frac{\log_2 a}{\log_2 b}$

$$h_{max}(m, N) = \left\lceil \log_{\lfloor m/2 \rfloor} \left(\frac{N+1}{2} \right) \right\rceil + 1$$

$$h_{max}(m, N) \leq \log_{\lfloor m/2 \rfloor} \left(\frac{N+1}{2} \right) + 1$$

$$h_{max}(m, N) \leq \frac{\log_m \left(\frac{N+1}{2} \right)}{\log_m \lfloor m/2 \rfloor} + 1 = \frac{\log_2 \left(\frac{N+1}{2} \right)}{\log_2 \lfloor m/2 \rfloor} + 1$$

□ Se m é par, $h_{max}(m, N) \leq \frac{\log_2 \left(\frac{N+1}{2} \right)}{\log_2 \frac{m}{2}} + 1 = \frac{\log_2 \left(\frac{N+1}{2} \right)}{\log_2(m)-1} + 1$

□ Se m é ímpar, $h_{max}(m, N) \leq \frac{\log_2 \left(\frac{N+1}{2} \right)}{\log_2 \frac{m+1}{2}} + 1 = \frac{\log_2 \left(\frac{N+1}{2} \right)}{\log_2(m+1)-1} + 1$

O custo de ter uma árvore com apenas 50% de ocupação...

□ Seja $\delta_m = m \bmod 2 = \begin{cases} 0 & \text{se } m \text{ é par} \\ 1 & \text{se } m \text{ é ímpar} \end{cases}$

□ Portanto

- $\lceil m/2 \rceil = \frac{m+\delta_m}{2}$

- $\log_2 \lceil m/2 \rceil = \log_2 \left(\frac{m+\delta_m}{2} \right) = \log_2(m + \delta_m) - 1$

- $h_{\max}(m, N) \leq \frac{\log_2 \left(\frac{N+1}{2} \right)}{\log_2 \frac{m+\delta_m}{2}} + 1 = \frac{\log_2 \left(\frac{N+1}{2} \right)}{\log_2(m+\delta_m) - 1} + 1$

□ Como $\delta_m \in \{0,1\}$, para qualquer $m \geq 3$ temos que $m + \delta_m \approx m$

O custo de ter uma árvore com apenas 50% de ocupação...

$$h_{min}(m, N) = \lceil \log_m(N + 1) \rceil \geq \log_m(N + 1) = \frac{\log_2(N + 1)}{\log_2 m}$$

$$h_{max}(m, N) = \left\lceil \log_{\lceil m/2 \rceil} \left(\frac{N + 1}{2} \right) \right\rceil + 1 \leq \frac{\log_2 \left(\frac{N + 1}{2} \right)}{\log_2 \frac{m + \delta_m}{2}} + 1 = \frac{\log_2 \left(\frac{N + 1}{2} \right)}{\log_2(m + \delta_m) - 1} + 1$$

$$\therefore \mathcal{O}(h_{min}) = \mathcal{O}(h_{max}) = \mathcal{O} \left(\frac{\log_2 N}{\log_2 m} \right) = \mathcal{O}(\log_m N)$$

- ❑ Há perda mínima: ter um nó com metade da capacidade ($\lceil m/2 \rceil$) em vez da capacidade total (m) custa exatamente 1 bit de informação no fator de ramificação (achatamento)
- ❑ Assim, o impacto no fator de achatamento, assumindo o fator de achatamento ideal de $\log_2 m$, no pior caso ele se torna $\log_2 m - 1$
- ❑ Exemplo:
 - se $m = 1024$, o fator de achatamento ideal é 10
 - Se a árvore estiver na ocupação mínima (50%), o fator é 9
 - A diferença na altura da árvore entre estar 100% cheia ou 50% cheia é de apenas $\frac{10}{9} = 1.11 \dots$ (cerca de 11%)

Ordem de Grandeza $O(h_{max})$

$$O(h_{max}) = O\left(\frac{\log_2 N}{\log_2 m}\right) = O(\log_m N)$$

- ❑ Ao expandirmos os termos para a base 2, a altura pode ser vista como o quociente entre a **complexidade do conjunto de dados** e a **capacidade de ramificação do nó**
 - Numerador $\log_2 N$ representa quantos bits de informação existem no total de chaves. É a altura de uma árvore binária perfeitamente balanceada
 - Denominador $\log_2 m$ representa o número bits de informação em cada nó. É o fator de achatamento (ou de ramificação) que “achata” a altura da árvore
- ❑ Na prática, a ordem de grandeza nos diz que a altura da Árvore B é uma fração da altura de uma árvore binária
 - Para uma árvore binária ($m = 2$), o denominador é $\log_2 2 = 1$ A altura é simplesmente $\log_2 N$
 - Para uma Árvore B com $m = 1024$, o denominador é $\log_2 1024 = 10$. Assim, a Árvore B terá uma altura 10 vezes menor que a de uma árvore binária para o mesmo volume de chaves

Escolha do m

- ❑ Dessa forma, sabemos que Árvores B de alta ordem são desejáveis, pois resultam em uma redução de número de acessos ao disco, necessários para pesquisar um índice
- ❑ Para um índice com N entradas, uma árvore B de ordem $m = N + 1$ teria apenas um nível
- ❑ Como supomos que o índice é grande demais para ser acomodado na memória interna, essa escolha de m claramente não seria razoável
- ❑ Consequentemente, um único nó representando o índice não caberia na memória principal para ser processado
- ❑ Para encontrar uma escolha razoável de m , precisamos nos lembrar que estamos interessados na minimização do tempo total necessário para pesquisar o valor X na árvore B

Escolha do m

- ❑ Suponhamos que cada nó da árvore B de ordem m é de tamanho fixo e suficientemente grande para acomodar n , A_0 e $(m - 1)$ valores para (K_i, A_i, B_i) , $1 \leq i < m$
- ❑ Se os K_i 's ocupam, no máximo, α bytes e ambos A_i e B_i cada um β bytes então o tamanho de um nó será de aproximadamente $m \cdot (\alpha + 2\beta)$ bytes
- ❑ O tempo t para ler um nó será então:
 - $t = t_p + t_l + m \cdot (\alpha + 2\beta) \cdot t_b = a + b \cdot m$, onde
 - $a = t_p + t_l$ = tempo de posicionamento + tempo de latência
 - $b = (\alpha + 2\beta) \cdot t_b$ e t_b é o tempo de transmissão por byte
- ❑ Se a busca binária for usada para conduzir a pesquisa da linha 6 do algoritmo $mSearch$ então o tempo de processamento interno por nó será $c \cdot \log_2 m + d$, para algumas constantes c e d
- ❑ Desse modo, o tempo total de processamento por nó é
 - $\tau = a + b \cdot m + c \cdot \log_2 m + d$

Escolha do m

- ❑ Desse modo, o tempo total de processamento por nó é
 - $\tau = a + b \cdot m + c \cdot \log_2 m + d$
- ❑ O número de níveis h , para um índice com N entradas, é limitado por
 - $h \leq \log_{\lceil m/2 \rceil} \left(\frac{N+1}{2} \right) + 1$
- ❑ Logo, o tempo máximo de pesquisa é $T = h \cdot \tau$
$$T = h \cdot \tau = \left(\log_{\lceil m/2 \rceil} \left(\frac{N+1}{2} \right) + 1 \right) \cdot (a + b \cdot m + c \cdot \log_2 m + d)$$
- ❑ Termo de I/O: $h \cdot (a + b \cdot m)$
 - Este é o "gargalo". O valor de a (latência/posicionamento) é geralmente ordens de magnitude maior que o processamento na memória principal
 - Note que aumentar m diminui h , mas aumenta o tempo de leitura do nó ($b \cdot m$)
- ❑ Termo de Processamento Interno (CPU): $h \cdot (c \cdot \log_2 m + d)$
 - Como a busca interna é binária, mesmo para um m grande, este tempo é muito pequeno

Escolha do m

- ❑ Em termos de complexidade de tempo, se considerarmos as constantes fundidas para uma análise de comportamento, a função segue este padrão

$$T(m, N) \approx \left(\frac{\log_2 N}{\log_2 m} \right) \cdot (a + b \cdot m + c \cdot \log_2 m + d)$$

- ❑ Se m for muito pequeno, a altura h cresce drasticamente e o tempo total explode devido aos múltiplos acessos de posicionamento de disco (a)
- ❑ Se m for muito grande, a altura h fica mínima, mas o nó fica tão grande que o tempo de transmissão ($b \cdot m$) e a busca interna começam a pesar
- ❑ O ponto ideal de m é geralmente aquele em que o tamanho do nó ($m \cdot (\alpha + 2\beta)$) se iguala ao tamanho de uma página de disco (ou bloco), que costuma ser 4KB ou 8KB, o que minimiza o desperdício e otimiza a leitura física

Escolha do m

- A curva do tempo total T em função da ordem m possui um formato de "U" (parábola assimétrica)
 - Isso ocorre porque ela é o resultado do equilíbrio entre duas forças opostas que dependem de m
- Altura da Árvore ($h \approx 1/\log m$)
 - Conforme m aumenta, a altura cai rapidamente no início e depois estabiliza
 - Isso tende a diminuir o tempo total, pois reduz o número de vezes que o disco é acessado
- Tempo por Nó ($\tau \approx a + b \cdot m$)
 - Conforme m aumenta, o tempo para ler e processar cada nó individual aumenta linearmente (devido ao tamanho do registro e transmissão)
 - Isso tende a aumentar o tempo total

Escolha do m

- Ao multiplicar esses dois componentes ($T = h \cdot \tau$), observamos três fases distintas na curva:
- Lado Esquerdo (Valores baixos de m): O tempo T é altíssimo. Como m é pequeno, a árvore é muito alta. O custo fixo de posicionamento de disco (a) é pago muitas vezes. Aqui, a curva descende agressivamente conforme m aumenta
 - O Ponto de Mínimo (O Vale): É o ponto de eficiência máxima, no qual o ganho de reduzir a altura da árvore não compensa mais o custo de ler nós cada vez maiores. Em sistemas reais, esse ponto geralmente coincide com o tamanho da página do sistema operacional (ex: 4KB)
 - Lado Direito (Valores altos de m): O tempo T volta a subir, mas de forma mais suave. A árvore já está com altura mínima (ex: $h = 2$ ou $h = 3$, então aumentar m não reduz mais o número de acessos ao disco, mas torna cada acesso individualmente mais lento e pesado

Escolha do m

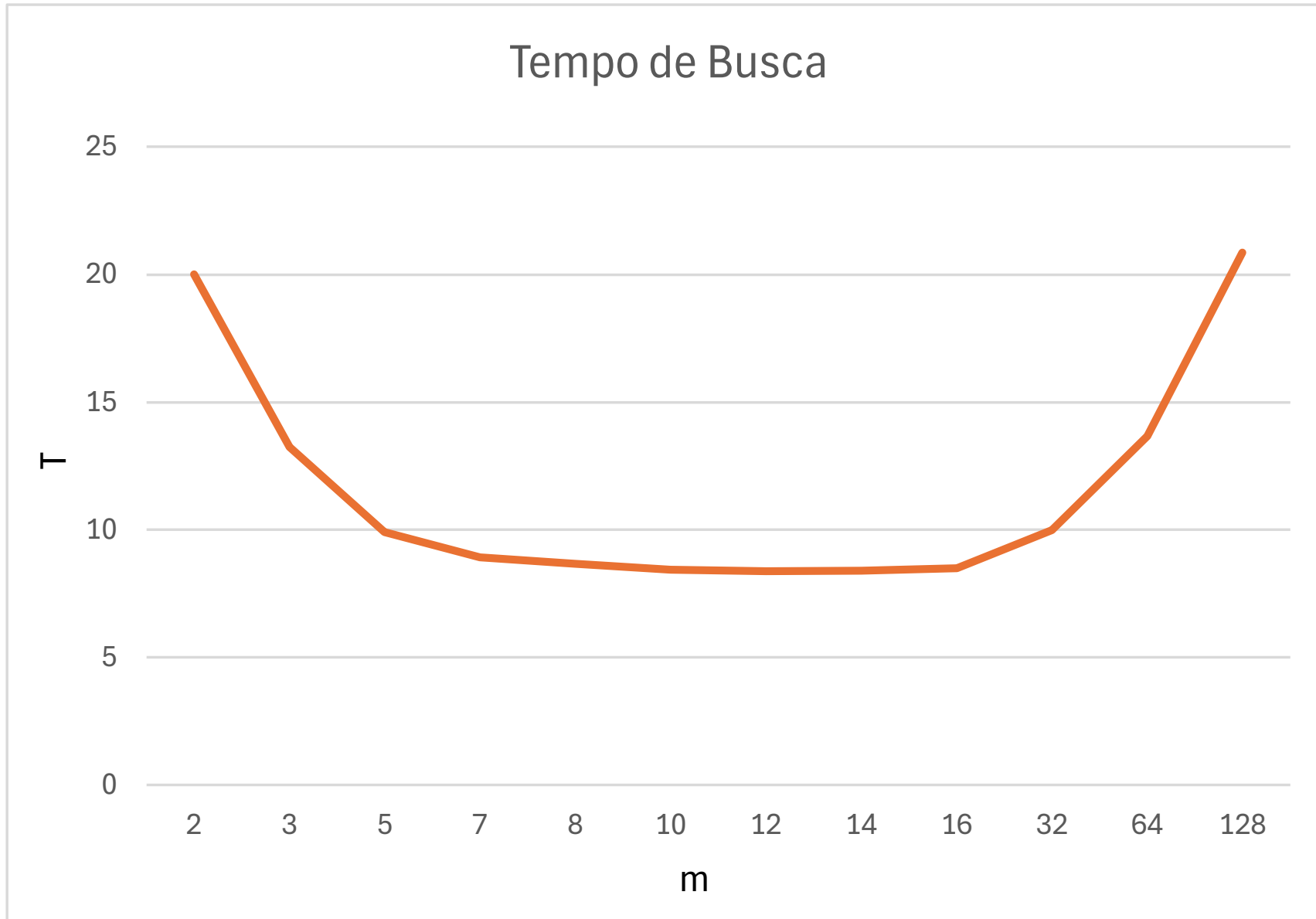
- ❑ Se ignorarmos as constantes menores, a curva se comporta proporcionalmente a

$$T(m) \propto \frac{a + b \cdot m}{\log_2 m}$$

- ❑ Assumindo que a unidade de disco disponível tem $t_p = 10$ ms e $t_l = 8$ ms, teremos $a = 18$ ms
- ❑ Assumindo que cada valor de chave tem, no máximo, 6 bytes e que cada A_i e B_i ocupam, cada um, 3 bytes, temos $\alpha = 6$ e $\beta = 3$
- ❑ Se a velocidade de transmissão for $t_b = 5 \cdot 10^{-3}$ ms/byte então $b = (\alpha + 2\beta) \cdot t_b = 0.06$ ms
- ❑ A equação anterior fica

$$T(m) \propto \frac{18 + 0.06 \cdot m}{\log_2 m}$$

Tempo de Busca para $T(m) \propto \frac{18+0.06 \cdot m}{\log_2 m}$



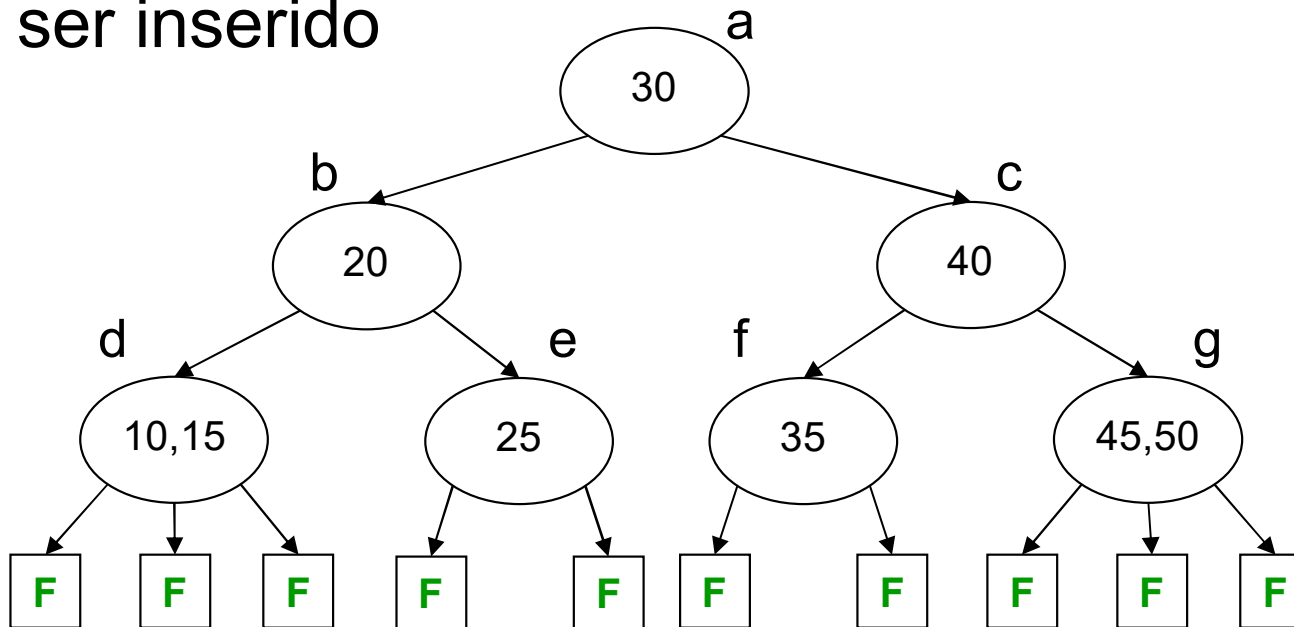
m	T(m)
2	20.00
3	13.25
5	9.91
7	8.91
8	8.67
10	8.43
12	8.37
14	8.40
16	8.50
32	10.00
64	13.67
128	20.86

Inserção

- ❑ Além de pesquisar o índice para encontrar um valor de chave específico X , queremos também inserir e eliminar as entradas
- ❑ A inserção deve ser conduzida de tal maneira, que a árvore conseguida depois da inserção seja também uma árvore do tipo B
- ❑ O algoritmo para fazer isso é conceitualmente mais simples do que o algoritmo de inserção correspondente para uma árvore AVL

Inserção $X = 38$

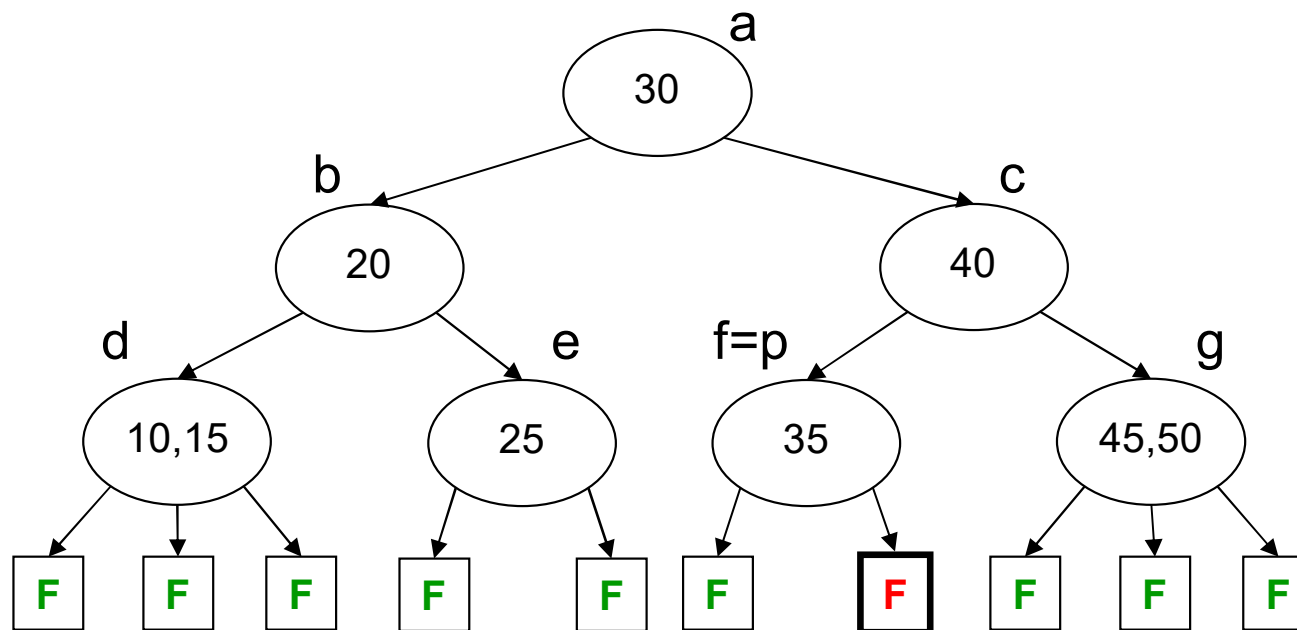
- ❑ Com o intuito de descobrir as operações necessárias para conduzir a inserção, considere a inserção de $X = 38$ na árvore
- ❑ Primeiro pesquisa-se a árvore para determinar onde 38 deve ser inserido



Inserção $X = 38$

R	W
3	

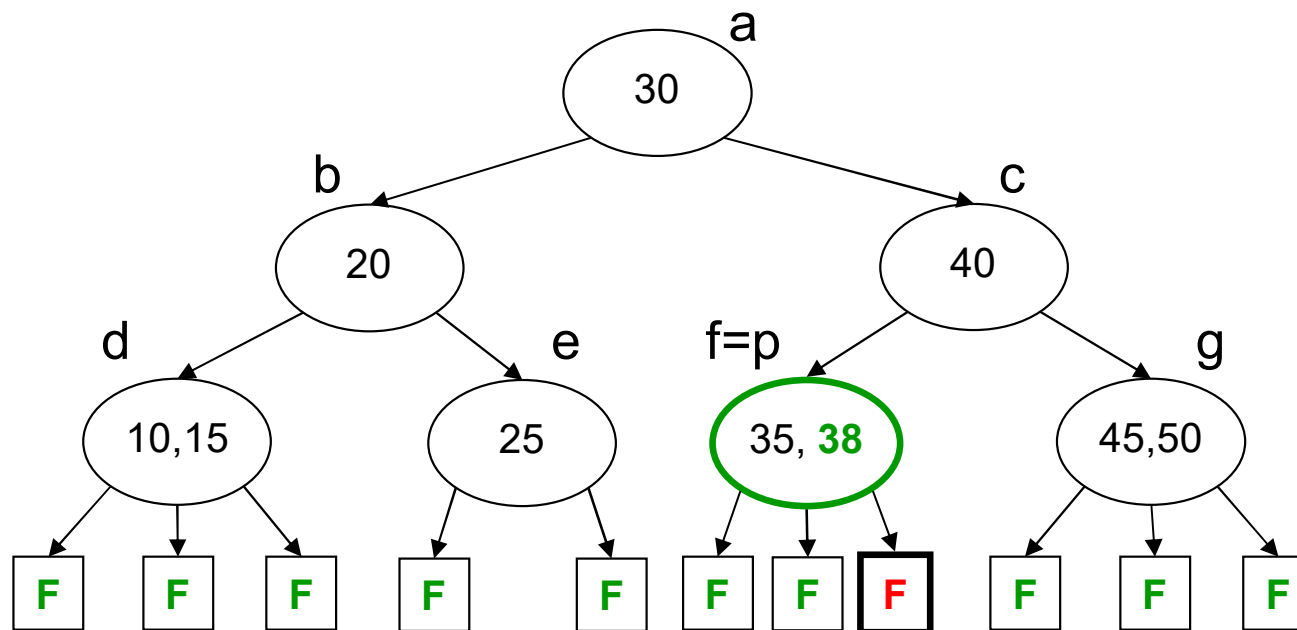
- ❑ O nó de falha que se alcança durante a pesquisa de $X = 38$ é o quarto da direita; o nó pai deste nó de falha é **f**
- ❑ O nó **f** contém apenas uma chave, e assim, tem espaço para outra



Inserção $X = 38$

R	W
3	1

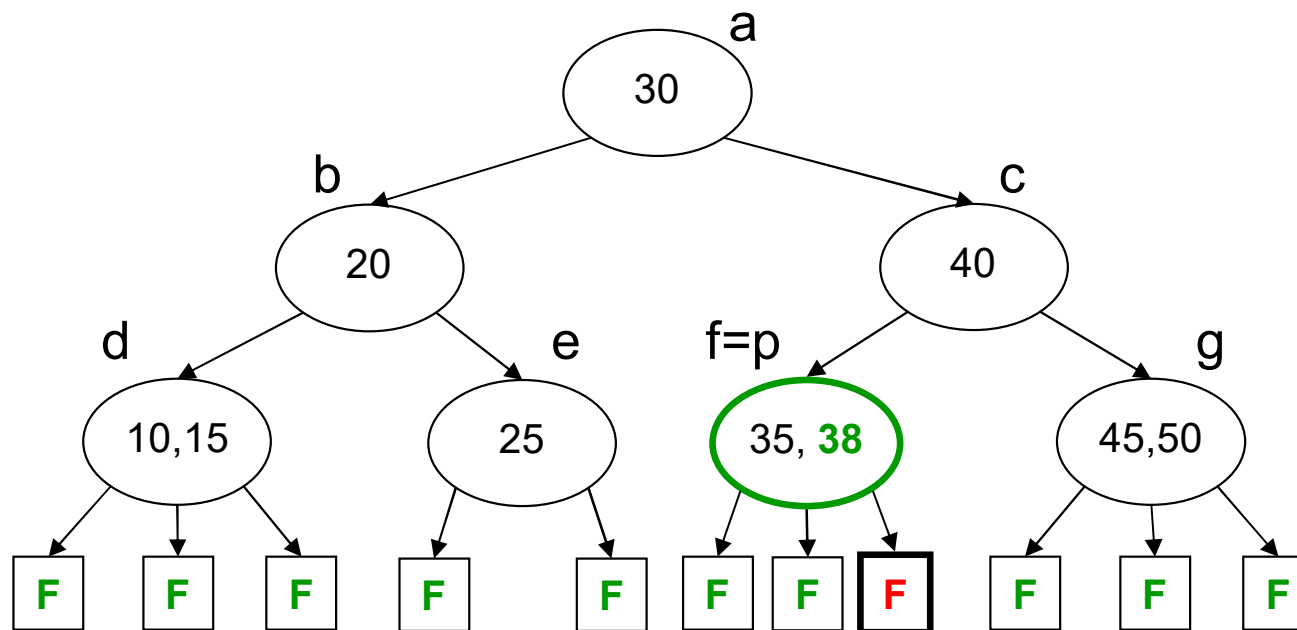
□ Logo, $X = 38$ pode ser inserido no nó **f**,
obtendo-se a árvore seguinte



Inserção X = 38

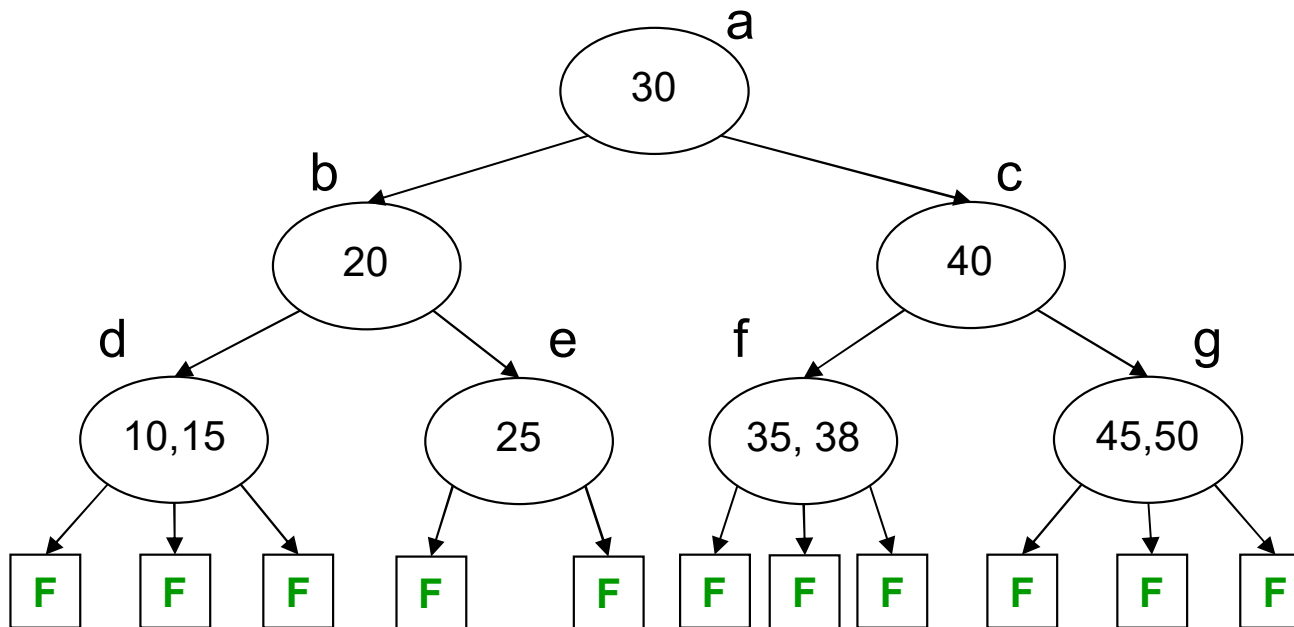
R	W
3	1

- ❑ Fazendo esta inserção, três nós **a**, **c** e **f** foram acessados no disco durante a pesquisa
- ❑ Além disso, o novo nó **f** precisa ser gravado de volta no disco
- ❑ A quantidade total de acessos é, portanto, quatro



Inserção $X = 55$

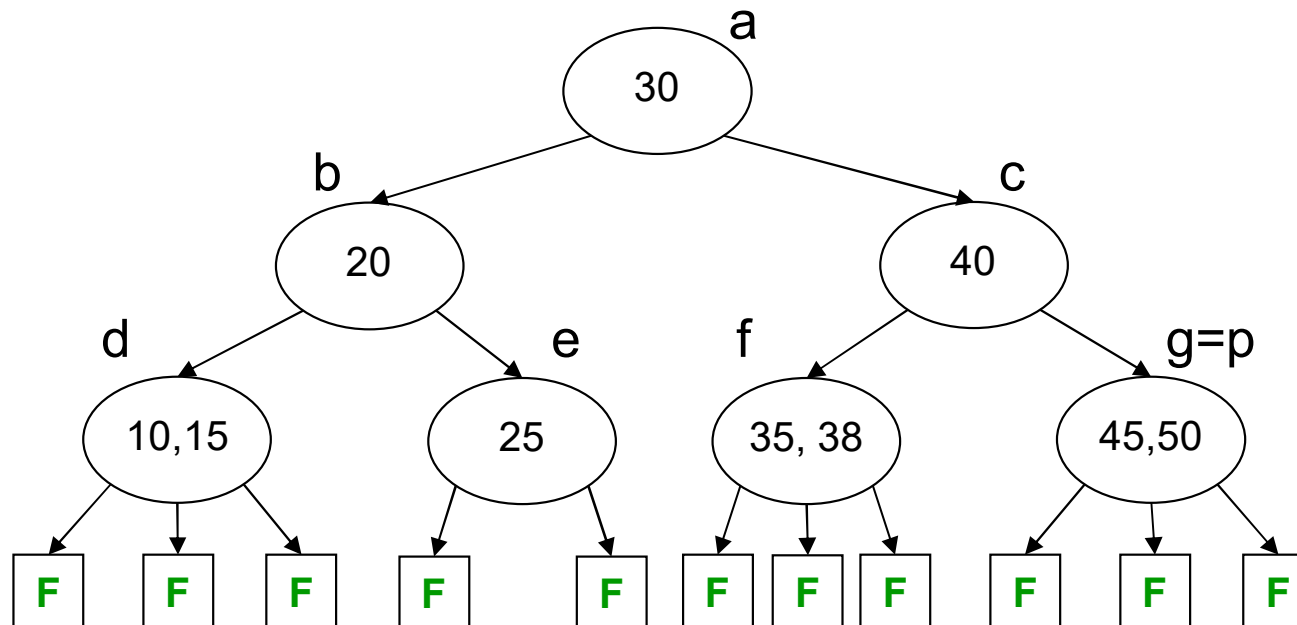
□ A seguir, vamos inserir $X = 55$



Inserção $X = 55$

R	W
3	

- ❑ A seguir, vamos inserir $X = 55$
- ❑ Uma pesquisa na árvore B indica que esta chave deve ser encaminhada para o nó **g**
- ❑ Não há espaço neste nó, pois ele já contém **m-1** chaves



Inserção (Eq.1)

- Inserindo simbolicamente o novo valor de chave em sua posição apropriada, em um nó que anteriormente tinha $m-1$ chaves, produzirá um nó P , com o seguinte formato
 - $p: m, A_0, (K_1, A_1), (K_2, A_2), \dots, (K_m, A_m)$
 - ou seja,
 - $p: m, A_0, (K_1, A_1), \dots, (K_{\lceil m/2 \rceil - 1}, A_{\lceil m/2 \rceil - 1}), (K_{\lceil m/2 \rceil}, A_{\lceil m/2 \rceil}), (K_{\lceil m/2 \rceil + 1}, A_{\lceil m/2 \rceil + 1}), \dots, (K_m, A_m)$
 - Com $K_i < K_{i+1}, 1 \leq i < m$
- Esse nó pode ser fracionado em dois nós p e q com os seguintes formatos (Equação 1)
 - $p: \lceil m/2 \rceil - 1, A_0, (K_1, A_1), \dots, (K_{\lceil m/2 \rceil - 1}, A_{\lceil m/2 \rceil - 1})$
 - $q: m - \lceil m/2 \rceil, A_{\lceil m/2 \rceil}, (K_{\lceil m/2 \rceil + 1}, A_{\lceil m/2 \rceil + 1}), \dots, (K_m, A_m)$
- O valor restante $K_{\lceil m/2 \rceil}$ e o novo nó q formam o par $(K_{\lceil m/2 \rceil}, q)$ e é feita uma tentativa de inserir este par no pai de p

Número de Chaves em p e q

- Somando-se o número de chaves de p
 - $\lceil m/2 \rceil - 1$
- Com o número de chaves em q
 - $m - \lceil m/2 \rceil$
- Temos, $\lceil m/2 \rceil - 1 + m - \lceil m/2 \rceil = m - 1$ chaves
 - Resta, portanto, uma chave, $K_{\lceil m/2 \rceil}$

p: m	A_0 (K_1, A_1) ... $(K_{\lceil m/2 \rceil - 1}, A_{\lceil m/2 \rceil - 1})$	$(K_{\lceil m/2 \rceil},$	$A_{\lceil m/2 \rceil})$ $(K_{\lceil m/2 \rceil + 1}, A_{\lceil m/2 \rceil + 1})$... (K_m, A_m)
	Novo p, com $\lceil m/2 \rceil - 1$ chaves	Chave a ser inserida no pai de p, juntamente com q	q, com $m - \lceil m/2 \rceil$ chaves

Número de Chaves em p e q

- ❑ Recorde que o número mínimo de chaves em qualquer nó (exceto a raiz) em uma árvore B é $\lceil m/2 \rceil - 1$
- ❑ O número de chaves em p é $\lceil m/2 \rceil - 1 \geq \lceil m/2 \rceil - 1$ (mínimo)
- ❑ Note também que o número mínimo de chaves em cada nó é
 - Se m é par, $\lceil \frac{m}{2} \rceil - 1 = \frac{m}{2} - 1$
 - Se m é ímpar, $\lceil \frac{m}{2} \rceil - 1 = \frac{m+1}{2} - 1 = \frac{m}{2} - \frac{1}{2}$
- ❑ O número de chaves em q é
 - Se m é par, $m - \lceil \frac{m}{2} \rceil = m - \frac{m}{2} = \frac{m}{2} \geq \frac{m}{2} - 1$ (mínimo)
 - Se m é ímpar, $m - \lceil \frac{m}{2} \rceil = m - \frac{m+1}{2} = \frac{m}{2} - \frac{1}{2} \geq \frac{m}{2} - \frac{1}{2}$ (mínimo)
- ❑ É óbvio que tanto p como q possuem $\leq (m - 1)$ chaves (máximo)

Inserção (Eq.1)

- Assumindo $m=3$, temos
 - $p: 3, A_0, (K_1, A_1), (K_2, A_2), (K_3, A_3)$
 - Com $K_i < K_{i+1}, 1 \leq i < 3$
- Esse nó pode ser fracionado em dois nós p e q com os seguintes formatos (Equação 1)
 - $p: 1, A_0, (K_1, A_1)$
 - $q: 1, A_2, (K_3, A_3)$
- O valor restante K_2 e o novo nó q formam o par (K_2, q) e é feita uma tentativa de inserir este par no pai de p

Inserção (Eq.1)

□ Assumindo $m=4$, temos

- $p: 4, A_0, (K_1, A_1), (K_2, A_2), (K_3, A_3), (K_4, A_4)$
- Com $K_i < K_{i+1}, 1 \leq i < 4$

□ Esse nó pode ser fracionado em dois nós p e q com os seguintes formatos (Equação 1)

- $p: 1, A_0, (K_1, A_1)$
- $q: 2, A_2, (K_3, A_3), (K_4, A_4)$

□ O valor restante K_2 e o novo nó q formam o par (K_2, q) e é feita uma tentativa de inserir este par no pai de p

Inserção (Eq.1)

□ Assumindo $m=5$, temos

- $p: 5, A_0, (K_1, A_1), (K_2, A_2), (K_3, A_3), (K_4, A_4), (K_5, A_5)$
- Com $K_i < K_{i+1}, 1 \leq i < 5$

□ Esse nó pode ser fracionado em dois nós p e q com os seguintes formatos (Equação 1)

- $p: 2, A_0, (K_1, A_1), (K_2, A_2)$
- $q: 2, A_3, (K_4, A_4), (K_5, A_5)$

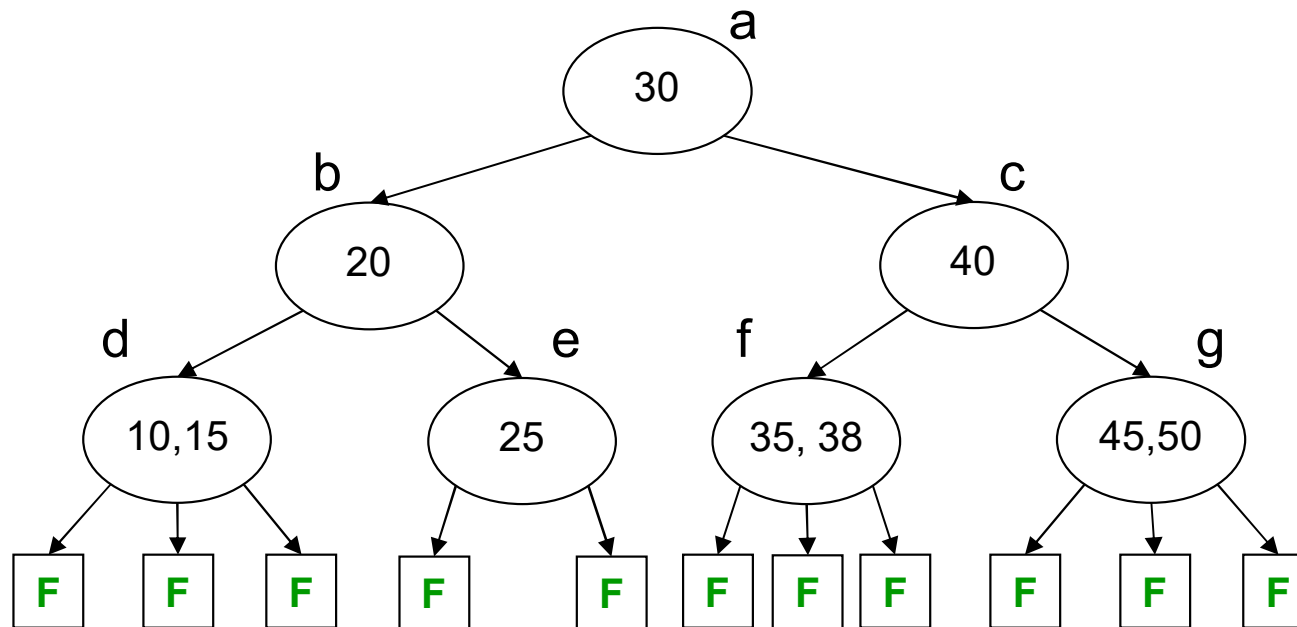
□ O valor restante K_3 e o novo nó q formam o par (K_3, q) e é feita uma tentativa de inserir este par no pai de p

Inserção $X = 55$

R	W
3	

❑ Considerando o nó **g**:

- 2, 0, (45,0), (50,0)



Inserção $X = 55$

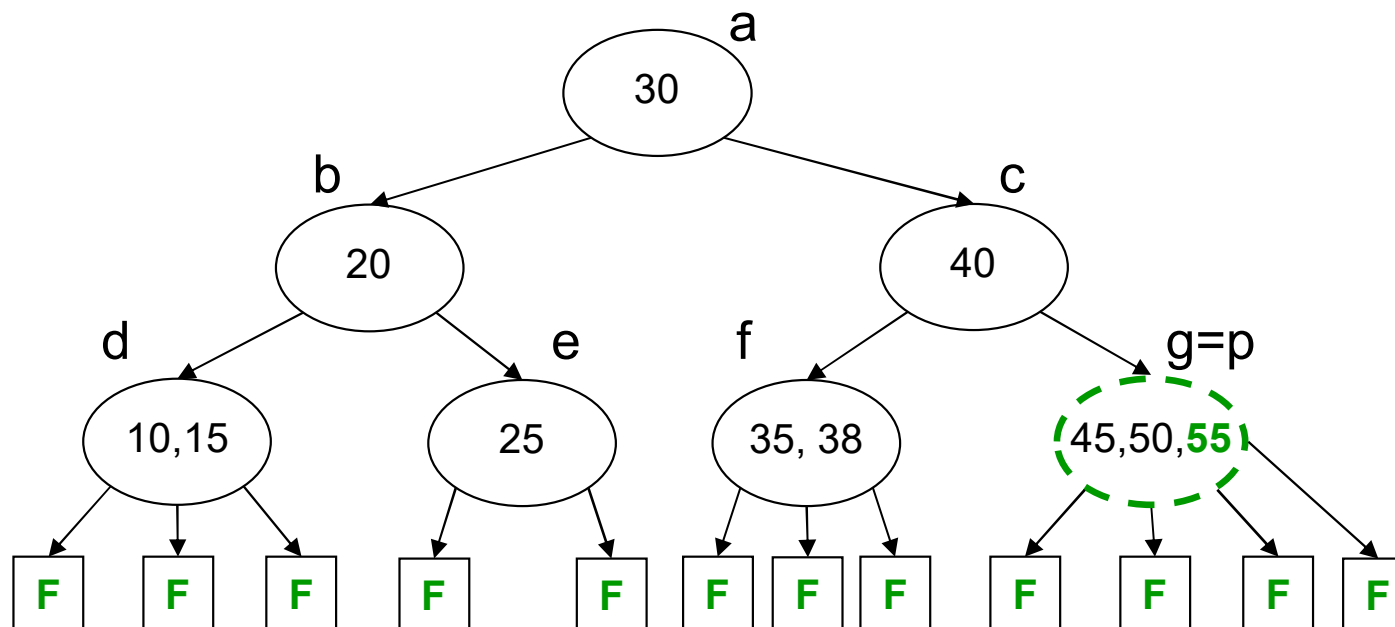
R	W
3	

❑ Considerando o nó **g**:

- 2, 0, (45,0), (50,0)

❑ Após a inserção simbólica de $X=55$, o nó **g** fica:

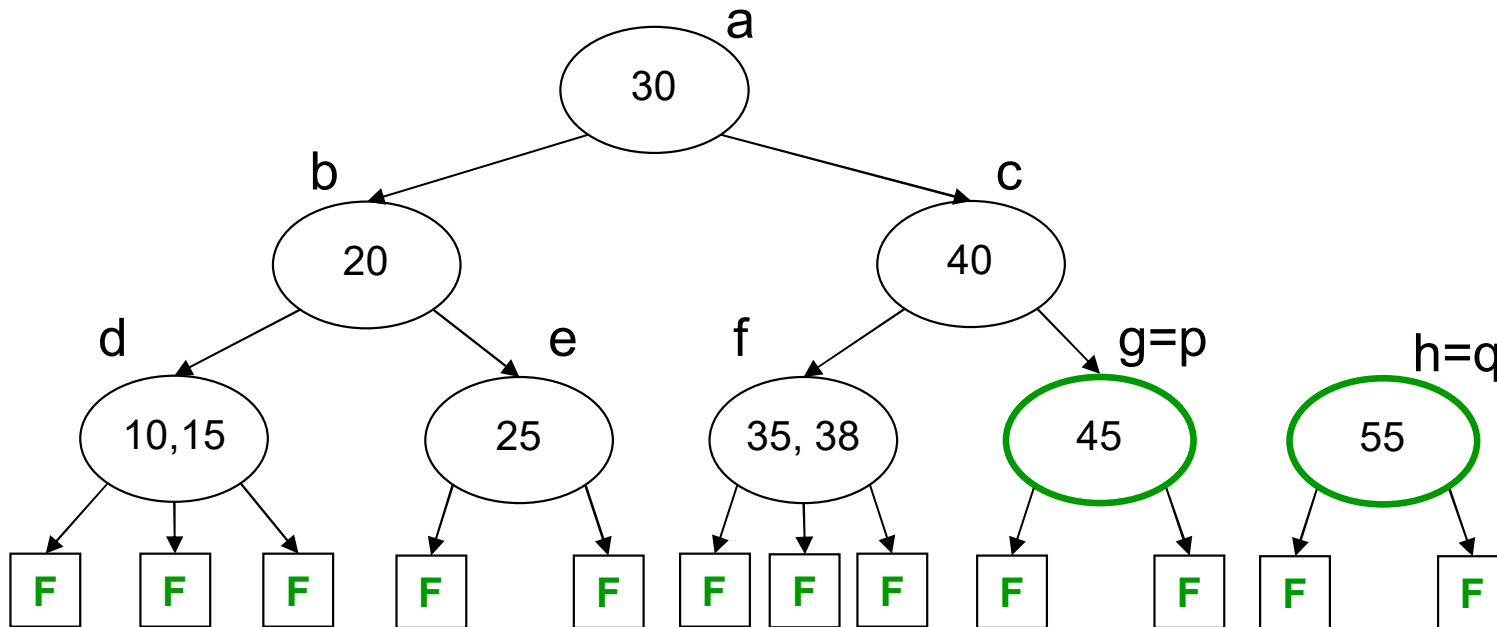
- 3, 0, (45,0), (50,0), (55, 0)



Inserção $X = 55$

R	W
3	2

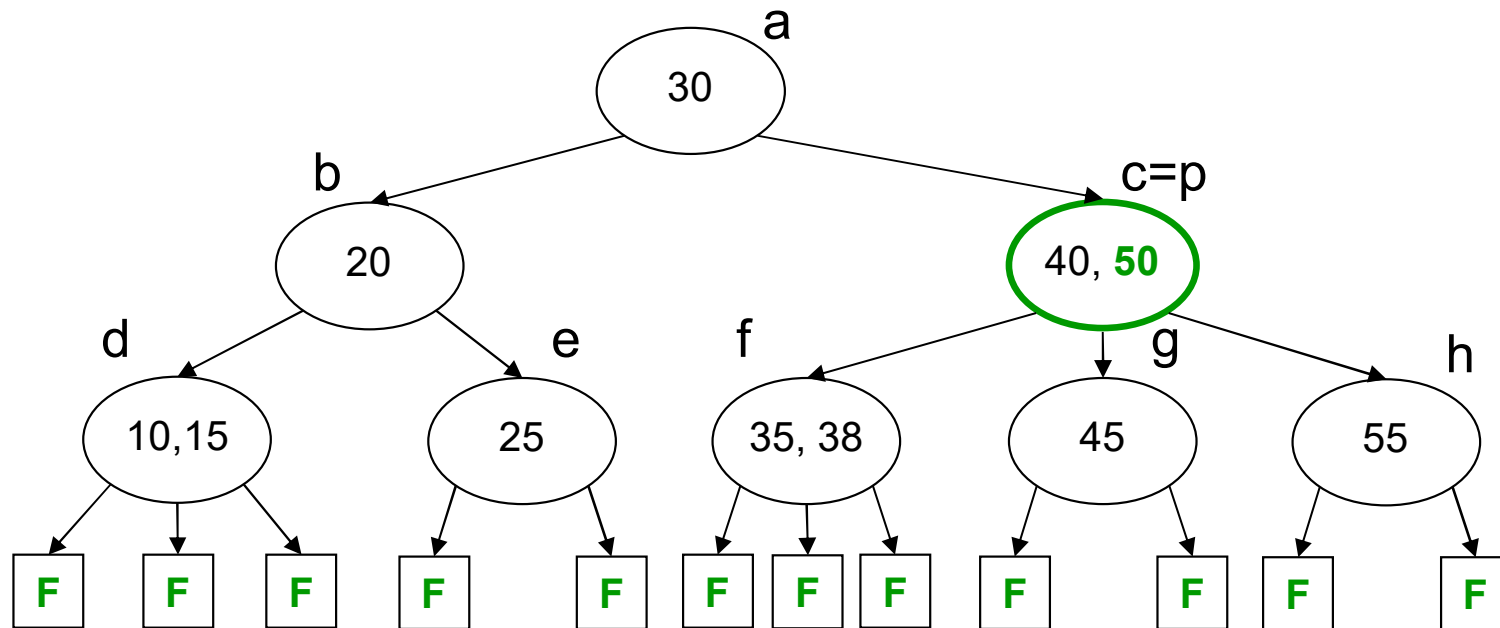
- ❑ Após a inserção simbólica de $X=55$, o nó **g** fica como:
 - 3, **0**, (**45,0**), (50,**0**), (**55, 0**)
- ❑ Portanto, o nó **g** precisa ser fracionado em:
 - **g**: 1, **0**, (**45,0**)
 - **h**: 1, **0**, (**55,0**)
- ❑ O par (50,**h**) deve ser inserido no pai de **g**, o nó **c**



Inserção $X = 55$

R	W
3	3

- Como o nó **c** tem apenas uma chave, a inserção do par (50, **h**) pode ser feita facilmente



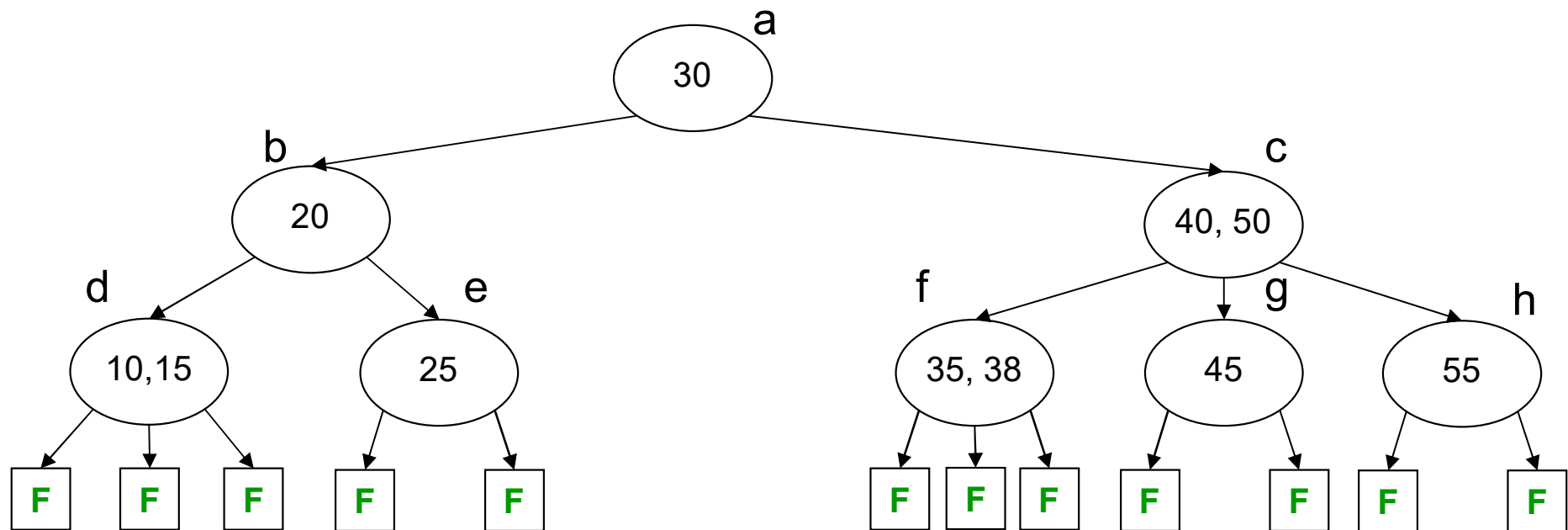
Inserção $X = 55$

R	W
3	3

- ❑ Desta vez foram feitos 3 acessos para determinar que $X = 55$ deve ser inserido em **g**
- ❑ Como o nó **g** foi alterado, precisa ser gravado no disco
- ❑ Assumindo que o nó **c** encontra-se ainda na memória interna, será necessário um outro acesso ao disco, para gravar o novo nó **c**
- ❑ Logo, o número total de acessos é 6

Exercício

- ❑ Inserir os valores 37, 5, 18 e 12 na árvore B, anotando quantos acessos a disco são necessários



Algoritmo de Inserção

- ❑ Pode-se verificar que as transformações de inserção, descritas anteriormente, preservam o índice como uma árvore do tipo B e cobrem todas as possibilidades
- ❑ O algoritmo de inserção resultante, **insertB**, assume que todos os nós, no caminho da raiz até o ponto de inserção, são empilhados
- ❑ Não tendo memória disponível bastará empilhar apenas os endereços e **pai(p)** poderá ser usado
- ❑ No algoritmo **insertB** o valor da chave X é inserido na árvore T de tipo B de ordem **m**
- ❑ Assume-me que T reside em disco e que a raiz da árvore B é o nó T

Algoritmo de Inserção

```
procedure insertB(T, X)
1  A ← 0; K ← X;                // (K,A) é o par a ser inserido
2  (p,i,achou) ← mSearch(T,X);   // p é o nó para inserção
3  if achou then return; endif   // X já está em T
4  while p ≠ 0 do
5      insira (K,A) nas posições apropriadas de p
      Seja o nó p resultante da forma: n, A0, (K1, A1), ..., (Kn, An)
6      if n ≤ m-1 then // nó resultante não é grande demais
          Escreva p para o disco; return;
      endif
      // p precisa ser fracionado
7      Sejam p e q definidos conforme Equação 1 (slide Inserção \(Eq.1\));
8      Escreva p e q para o disco;
9      K ← K[m/2]; A ← q; p ← pai(p);
10 endwhile
    // a nova raiz deve ser criada
11 crie um novo nó r com formato 1,T,(K,A)
12 T ← r;
13 Escreva T para o disco
end insertB
```

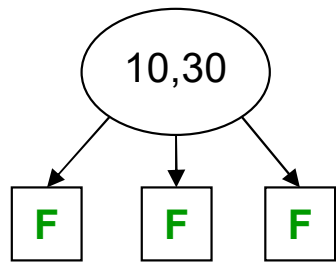
Análise insertB

- ❑ Se a árvore B tem h níveis então a pesquisa da linha 2 vai necessitar de h acessos, pois X não está em T
- ❑ Cada vez que um nó fraciona-se (linha 7) são feitos dois acessos ao disco (linha 8)
- ❑ Depois do fracionamento final é feito ainda um acesso, seja da linha 6 ou linha 13
- ❑ Sendo o número total de nós que se fracionam igual a k então o número total de acessos ao disco é $h+2*k+1$
- ❑ Este valor presume que exista a memória interna disponível, suficiente para manter todos os nós acessados durante a chamada de **mSearch** da linha 2
- ❑ Como no máximo um nó pode ser fracionado em cada um dos h níveis, $k \leq 1$ sempre (em cada nível)
- ❑ Portanto, o número máximo de acessos será $3*h+1$
- ❑ No pior caso, como em situações práticas, h não excederá a 4, o que significaria em torno de treze acessos necessários para fazer uma inserção

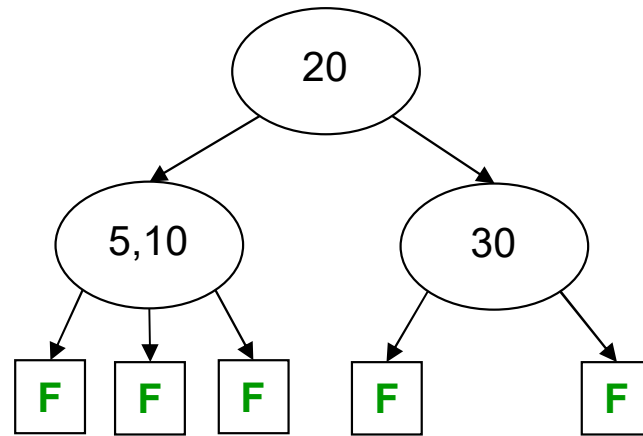
Análise insertB

- ❑ No caso de algoritmo **insertB**, o valor de pior caso de $3 \cdot h + 1$ acessos não reflete a história completa
- ❑ O número médio de acessos é consideravelmente inferior a isso
- ❑ Uma estimativa do número de acessos médio, necessário para uma inserção pode ser obtido se começarmos com uma árvore vazia e inserirmos nela **N** valores; então o número total de fracionamentos de nós é, no máximo, $(p-2)$, onde **p** é o número de nós não-de-falha na árvore B final, com **N** chaves
- ❑ Esta limitação superior de $(p-2)$ origina-se da observação de que cada vez que um nó é fracionado, cria-se pelo menos um nó adicional
- ❑ Quando a raiz é fracionada, criam-se dois nós adicionais
- ❑ O primeiro nó criado não tem origem de fracionamento e se uma árvore B tiver mais do que um único nó, então a sua raiz deve ter se fracionada pelo menos uma vez
- ❑ O valor $(p-2)$ é a melhor limitação superior possível de fracionamentos de nós, na criação de uma árvore B com **p** nós
 - **p** > 2 (repare que não existe uma árvore B com **p** = 2)

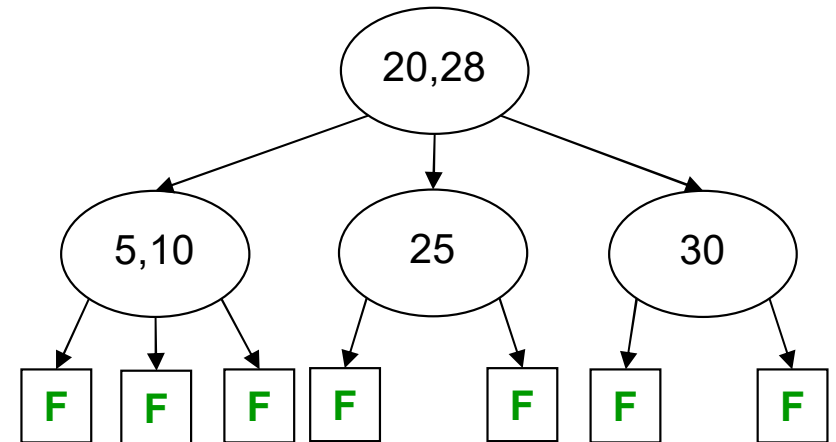
Análise insertB



$p=1$
Número de
fracionamentos = 0



$p=3$
Número de
fracionamentos = 1



$p=4$
Número de
fracionamentos = 2

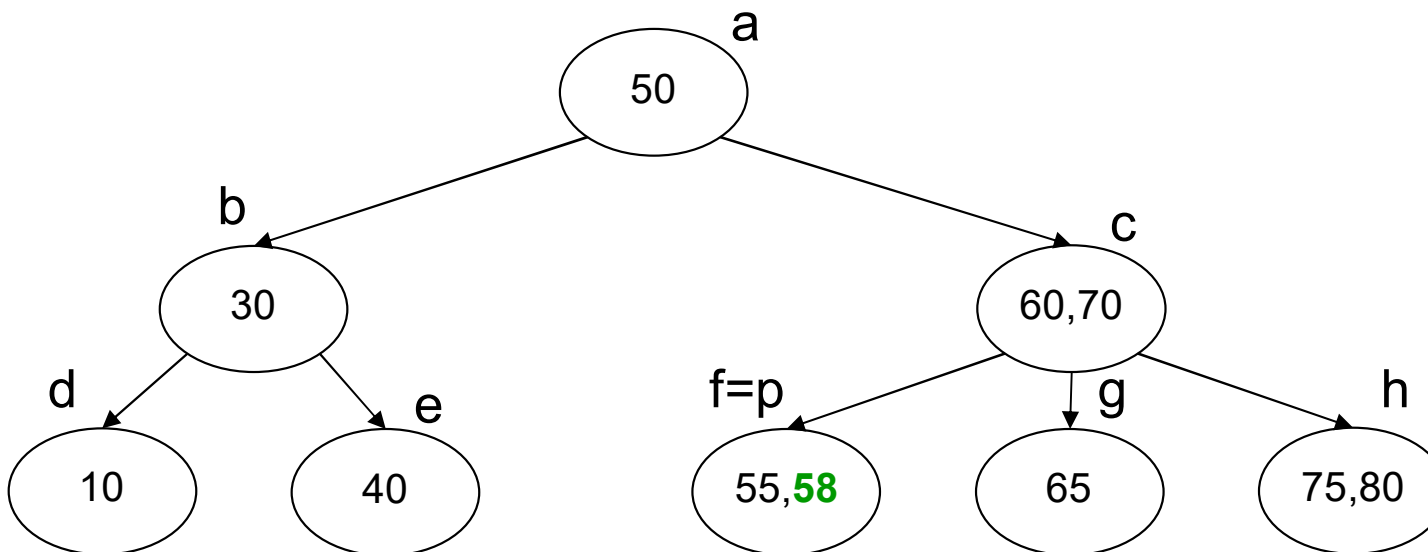
Análise insertB

- Uma árvore B de ordem m com p nós, tem pelo menos
 - $1 + (\lceil m/2 \rceil - 1) \cdot (p-1)$
- chaves, pois a raiz tem pelo menos um valor de chave e os nós restantes pelo menos $\lceil m/2 \rceil - 1$ chaves
- A quantidade média de fracionamentos s , pode ser determinada agora como
 - $s = (\text{quantidade total de fracionamentos})/N$
 - $s \leq (p-2) / (1 + (\lceil m/2 \rceil - 1) \cdot (p-1))$
 - $s < 1 / (\lceil m/2 \rceil - 1)$
- Para $m=200$ a quantidade de fracionamentos é inferior a $1/99$ por chave inserida
 - Logo, o número médio de acessos ao disco é de apenas $h + 2 \cdot s + 1 < h + 101/99 \approx h + 1$

Remoção $X = 58$

R	W
3	

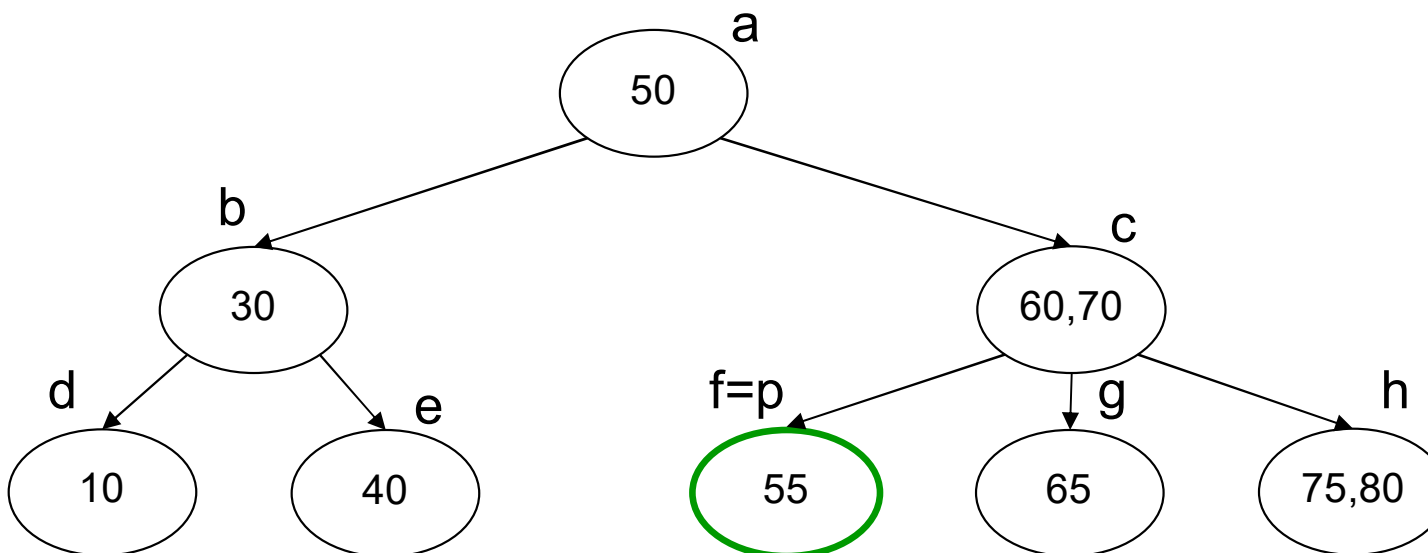
- ❑ Vamos analisar o problema de eliminar os valores dos nós folha
- ❑ A remoção do valor da chave $X = 58$...



Remoção $X = 58$

R	W
3	1

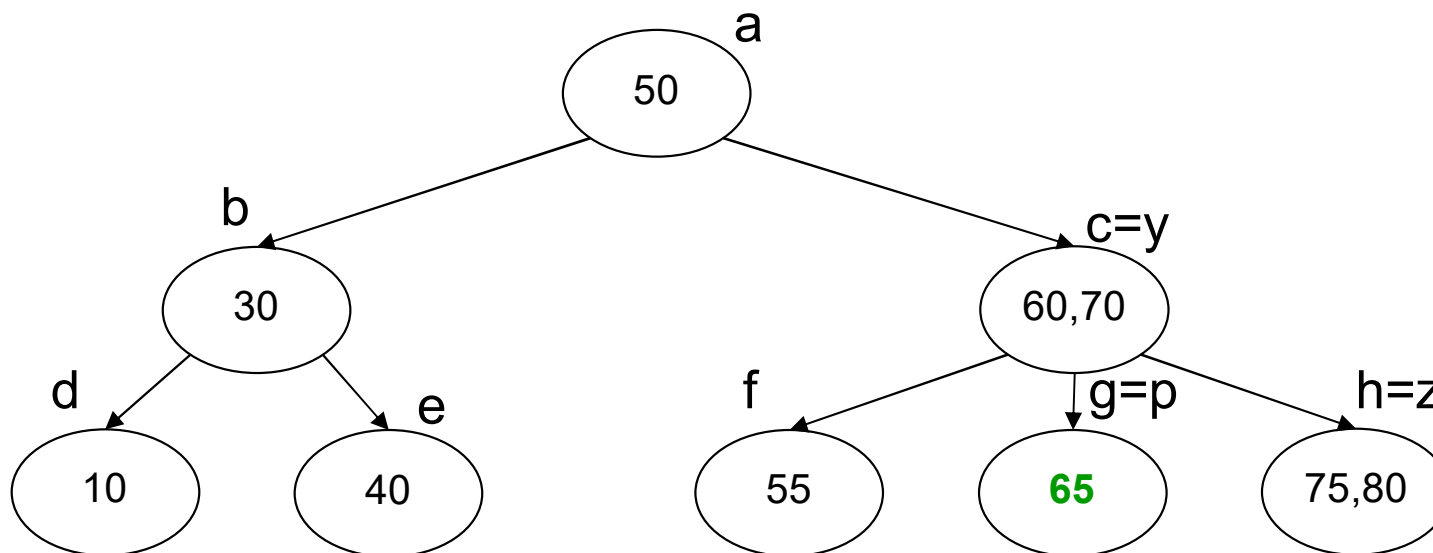
- ❑ Vamos analisar o problema de eliminar os valores dos nós folha
- ❑ A remoção do valor da chave $X = 58$ do nó **f** é fácil, já que a retirada deste valor de chave deixa o nó ainda com um outro valor de chave, que é o mínimo exigido para o nó diferente da raiz
- ❑ Uma vez que foi determinado que $X = 58$ está no nó **f**, o único acesso adicional será para regravar no disco o novo nó **f**



Remoção $X = 65$

R	W
3	

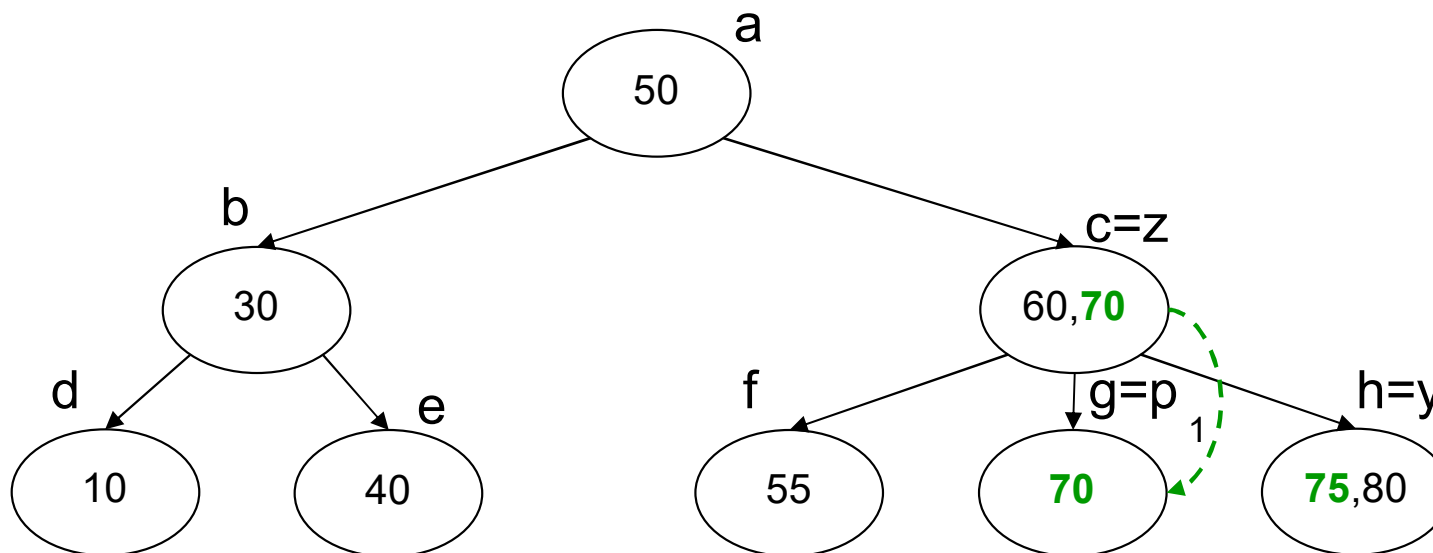
- A remoção de $X = 65$, do nó **g**, resulta em que o número de chaves restantes fique abaixo do mínimo exigido de $\lceil m/2 \rceil - 1$



Remoção $X = 65$

R	W
4	

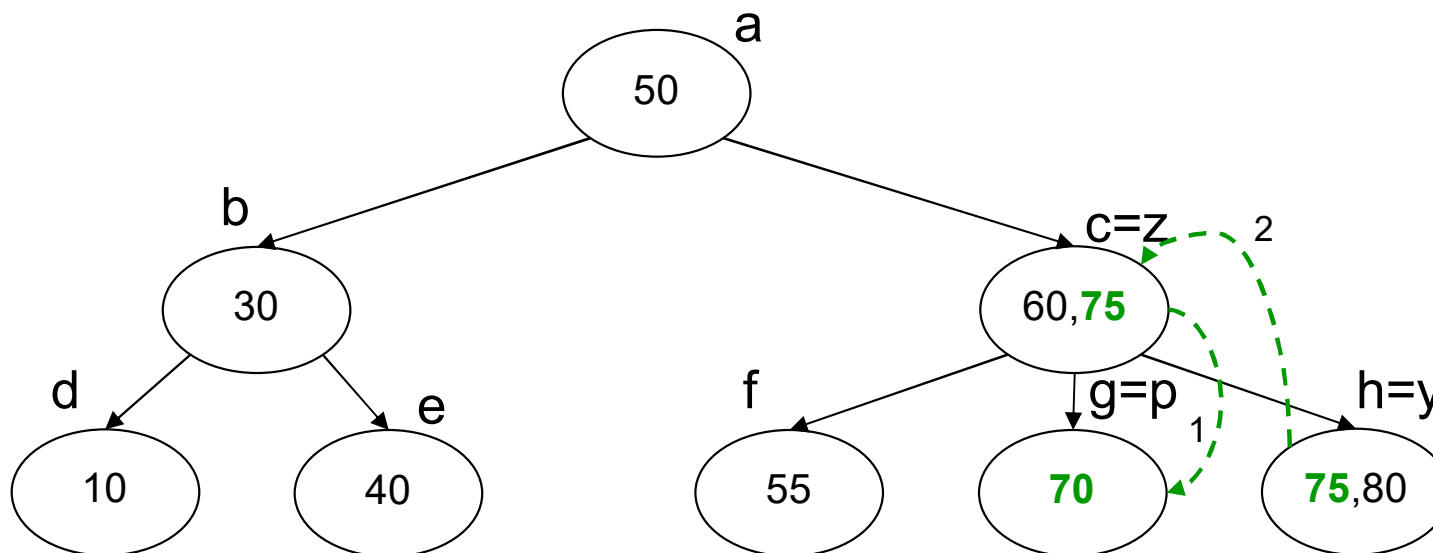
- ❑ A remoção de $X = 65$, do nó **g**, resulta em que o número de chaves restantes fique abaixo do mínimo exigido de $\lceil m/2 \rceil - 1$
- ❑ Um exame do irmão direito mais próximo **h**, do nó **g**, indica que ele tem $\geq \lceil m/2 \rceil$ chaves e então a chave de menor valor de **h**, isto é, 75 é movida para cima, para o nó pai **c**; antes disso, o valor 70 é movido do pai **c** para o filho **g**



Remoção $X = 65$

R	W
4	

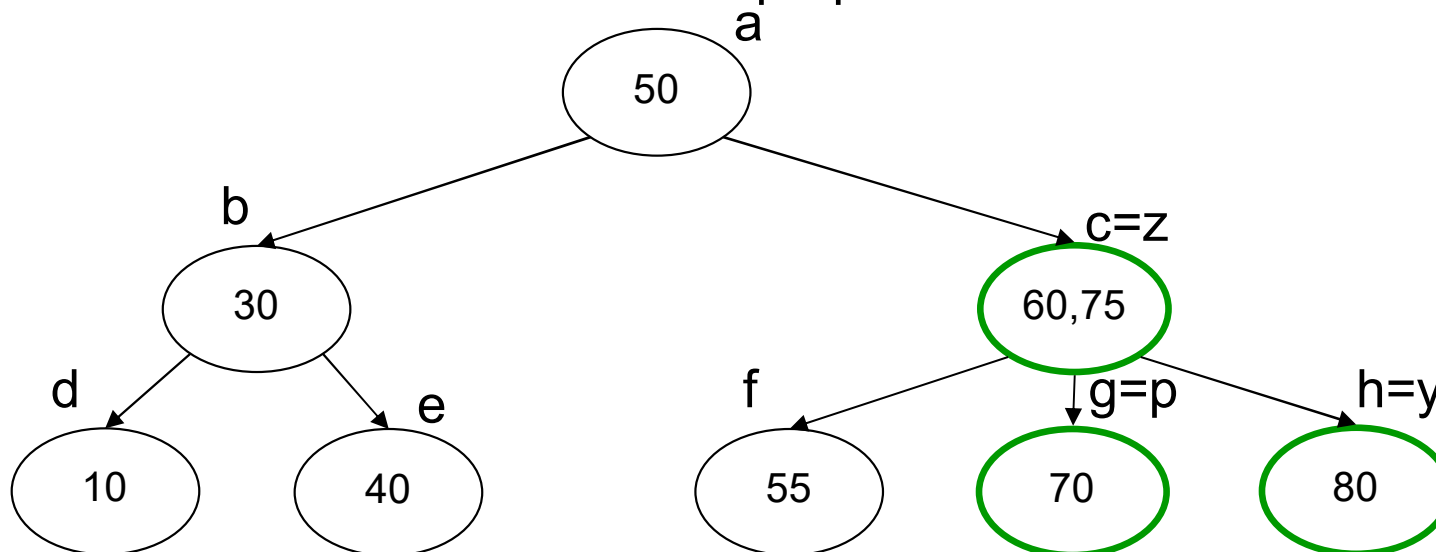
- ❑ A remoção de $X = 65$, do nó **g**, resulta em que o número de chaves restantes fique abaixo do mínimo exigido de $\lceil m/2 \rceil - 1$
- ❑ Um exame do irmão direito mais próximo **h**, do nó **g**, indica que ele tem $\geq \lceil m/2 \rceil$ chaves e então a chave de menor valor de **h**, isto é, 75 é movida para cima, para o nó pai **c**; antes disso, o valor 70 é movido do pai **c** para o filho **g**



Remoção $X = 65$

R	W
4	3

- ❑ A remoção de $X = 65$, do nó **g**, resulta em que o número de chaves restantes fique abaixo do mínimo exigido de $\lceil m/2 \rceil - 1$
- ❑ Um exame do irmão direito mais próximo **h**, do nó **g**, indica que ele tem $\geq \lceil m/2 \rceil$ chaves e então a chave de menor valor de **h**, isto é, 75 é movida para cima, para o nó pai **c**; antes disso, o valor 70 é movido do pai **c** para o filho **g**
- ❑ Como resultado dessas transformações, o número de chaves é $> \lceil m/2 \rceil - 1$ tanto para **g** como para **h**; o nó **c** permanece sem alteração em número de chaves e a árvore B mantém as suas propriedades



Remoção $X = 65$

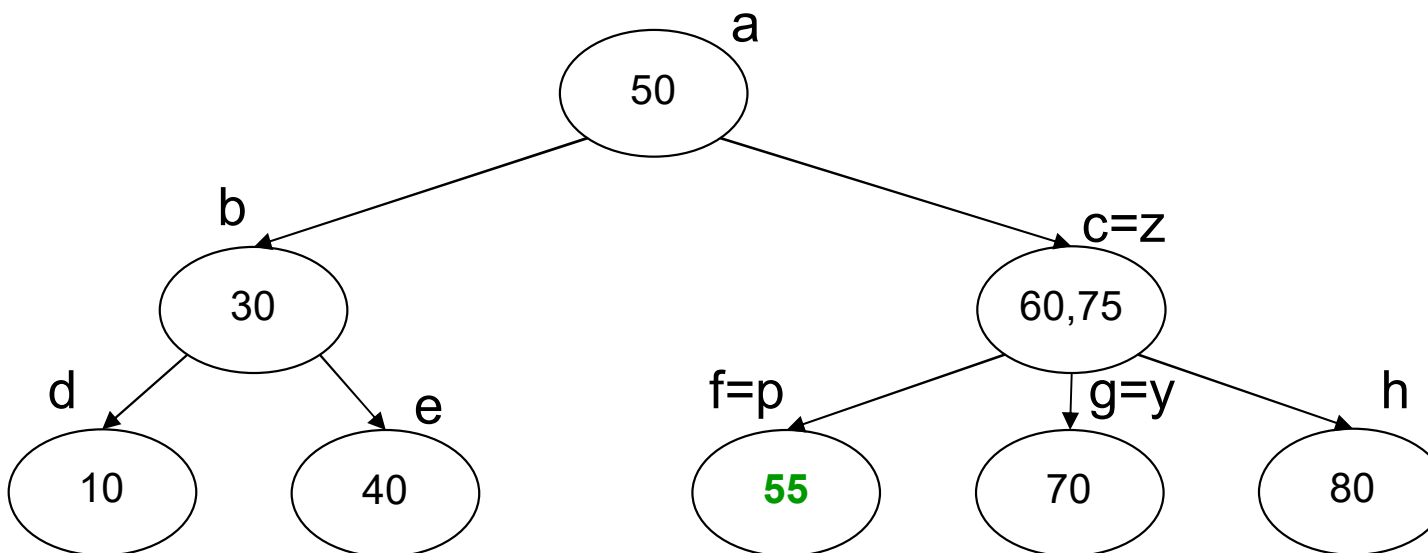
R	W
4	3

- ❑ Durante a pesquisa para encontrar $X = 65$ foram acessados os nós **a**, **c** e **g**
- ❑ Se todos esses três nós podem ser mantidos em memória, então pode ser determinado a partir do nó **c** o endereço de **h** como o irmão mais próximo de **g**; o nó **h** então é acessado
- ❑ Desde que três nós foram alterados (**g**, **c** e **h**), devem ser feitos mais três acessos para gravar novamente esses nós no disco
- ❑ Assim, precisam ser feitos mais quatro acessos, além dos acessos feitos para pesquisar a localização do $X = 65$, para efetuar a eliminação de $X = 65$
- ❑ Se **g** não tivesse um irmão direito próximo, ou se esse irmão tiver apenas a quantidade mínima de valores de chave, isto é, $\lceil m/2 \rceil - 1$ poder-se-ia fazer a mesma coisa com o irmão esquerdo mais próximo

Remoção $X = 55$

R	W
3	

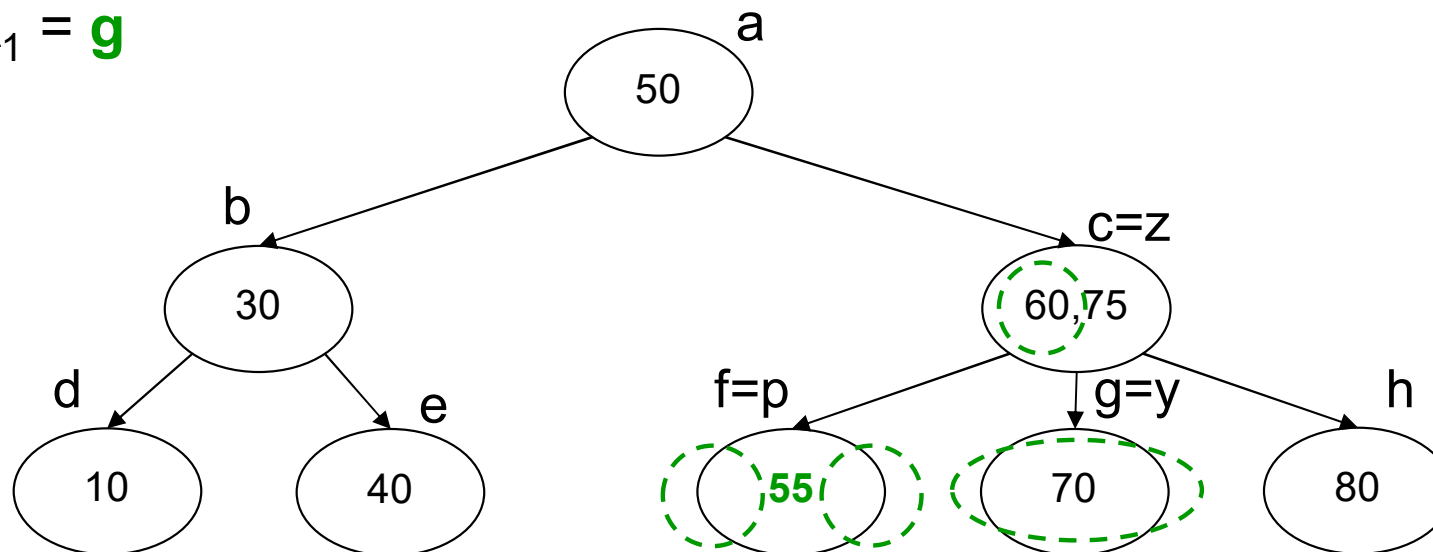
- ❑ Removendo $X = 55$ observamos que o irmão direito **g**, mais próximo de **f**, tem apenas $\lceil m/2 \rceil - 1$ chaves e que **f** não tem um irmão esquerdo próximo
- ❑ Consequentemente, é necessária uma transformação diferente



Remoção $X = 55$

R	W
4	

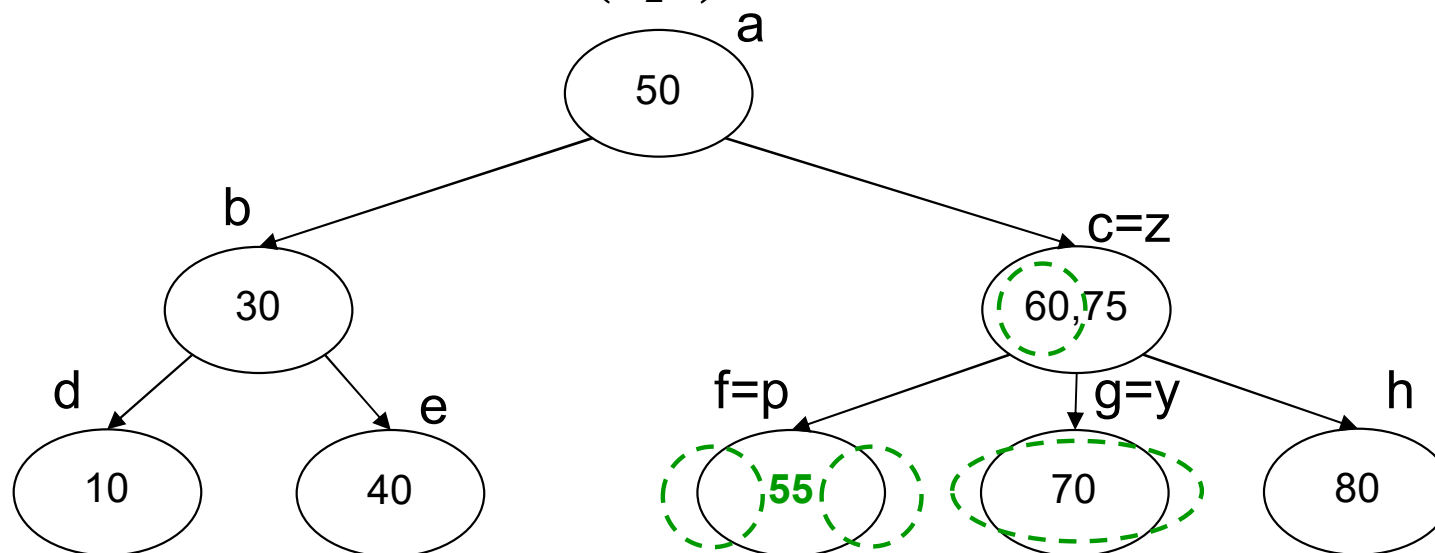
- Se o nó **c** (pai de **f**) estiver na forma
 - **c**: $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$ e
 - $A_i = \mathbf{g}$
- Então podemos combinar as chaves restantes de **f**, com as chaves de **g** e a chave K_i para formar um novo **f**
- No Exemplo
 - **c**: 2, **f**, (60, **g**), (75, **h**)
 - $A_1 = \mathbf{g}$



Remoção $X = 55$

R	W
4	

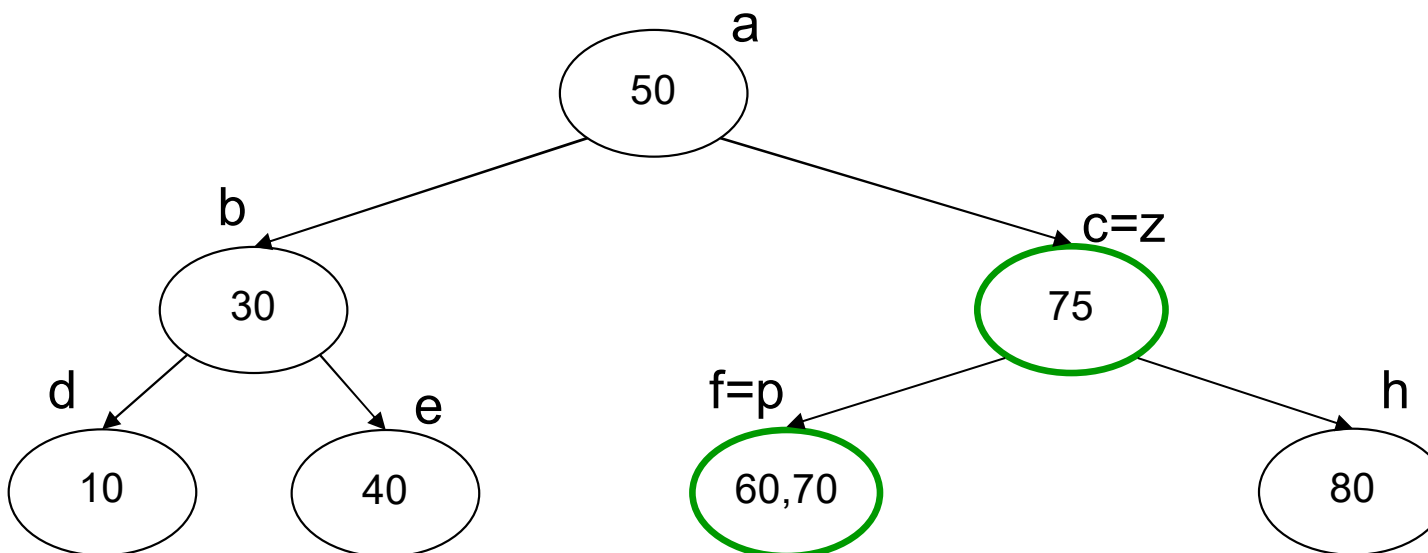
- Note que a combinação dos valores respeita o número máximo de $(m - 1)$ chaves por nó
- O novo nó **f** terá $(\lceil m/2 \rceil - 2) + (\lceil m/2 \rceil - 1) + 1 = 2\lceil m/2 \rceil - 2 \leq m - 1$ chaves
 - $(\lceil m/2 \rceil - 2)$ chaves restantes em **f**
 - $(\lceil m/2 \rceil - 1)$ chaves em **g**
 - 1 chave emprestada do pai **c**
- Se m é par, $2\lceil m/2 \rceil - 2 = 2\left(\frac{m}{2}\right) - 2 = m - 2 \leq m - 1$
- Se m é ímpar, $2\lceil m/2 \rceil - 2 = 2\left(\frac{m+1}{2}\right) - 2 = m + 1 - 2 = m - 1 \leq m - 1$



Remoção $X = 55$

R	W
4	2

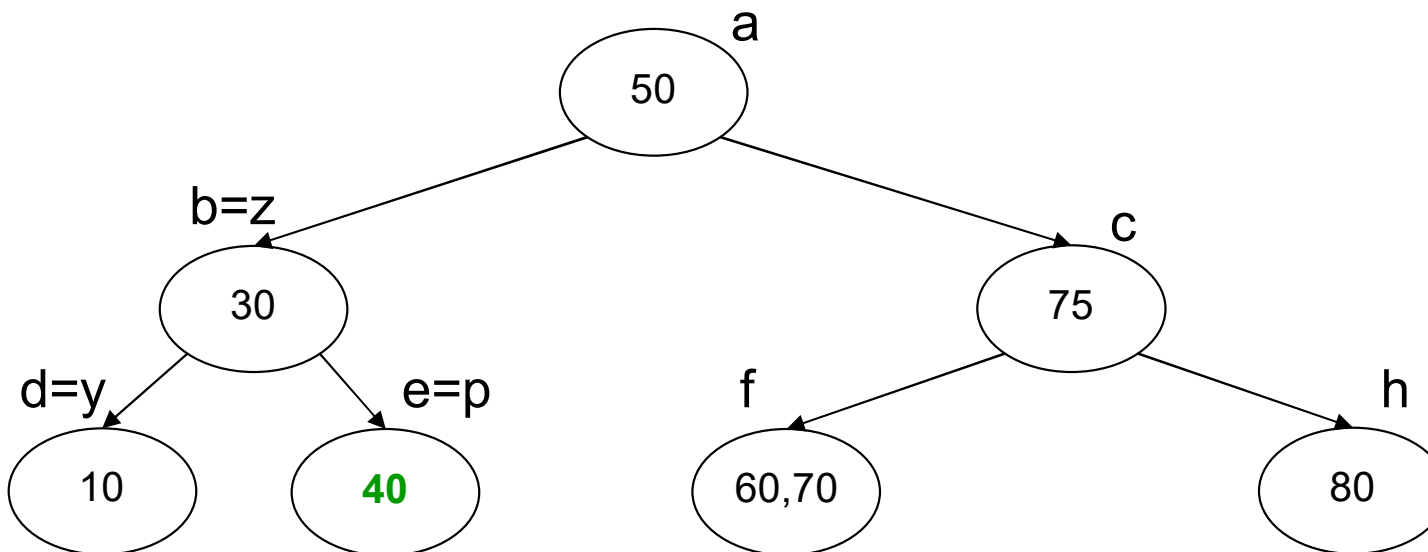
- São necessários três acessos adicionais (além daqueles necessários à busca)
 - um para constatar que **g** está pequeno demais e
 - um de cada para regravar **f** e **c**
- Note que o nó **g** ainda reside em disco e alguma estratégia de manutenção para esses nós removidos pode ser adotada para melhor aproveitamento do espaço



Remoção $X = 40$

R	W
3	

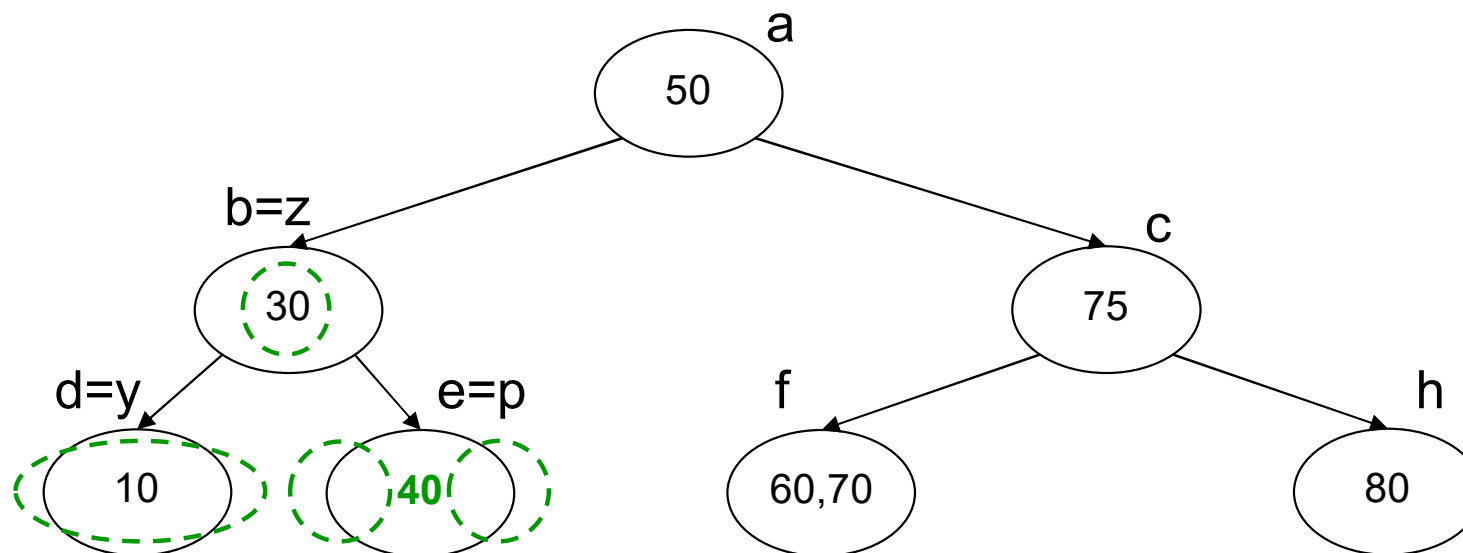
- ❑ A remoção de $X = 40$ deixa o nó **e** com $\lceil m/2 \rceil - 2$ chaves
- ❑ O seu irmão esquerdo **d** não tem nenhuma chave para emprestar



Remoção $X = 40$

R	W
4	

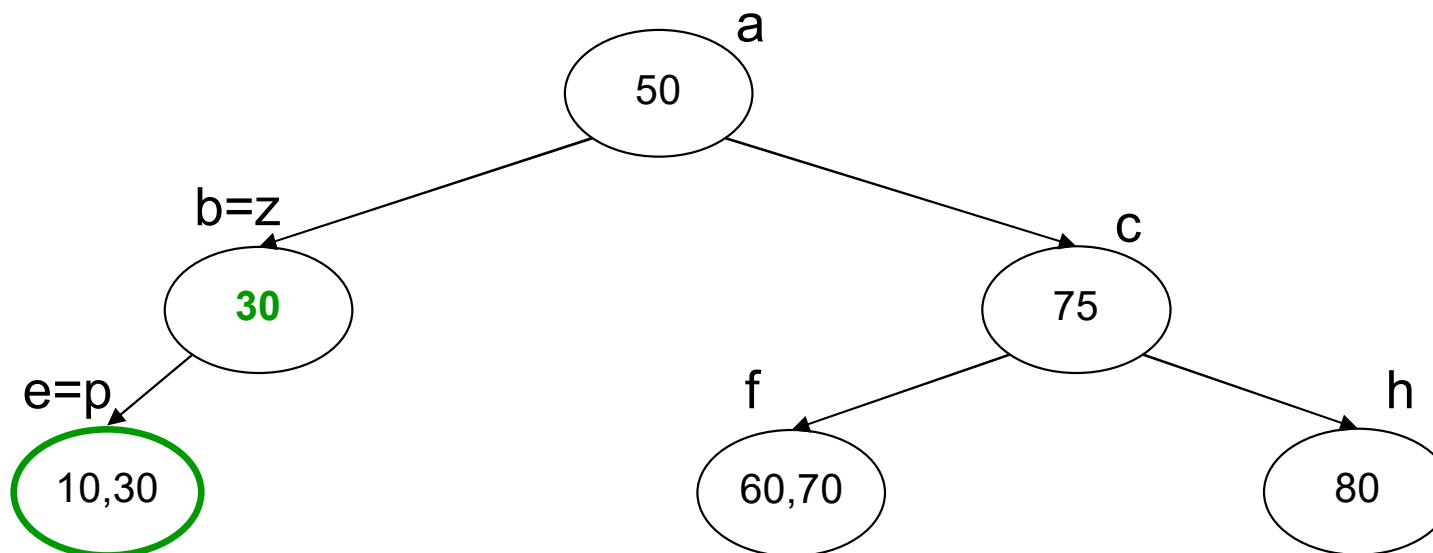
- Nesse caso, as chaves do nó **d** são combinadas com aquelas restantes do nó **e** e a chave 30 do nó pai **b** para obter o novo nó **e**, com os valores 10 e 30



Remoção $X = 40$

R	W
4	1

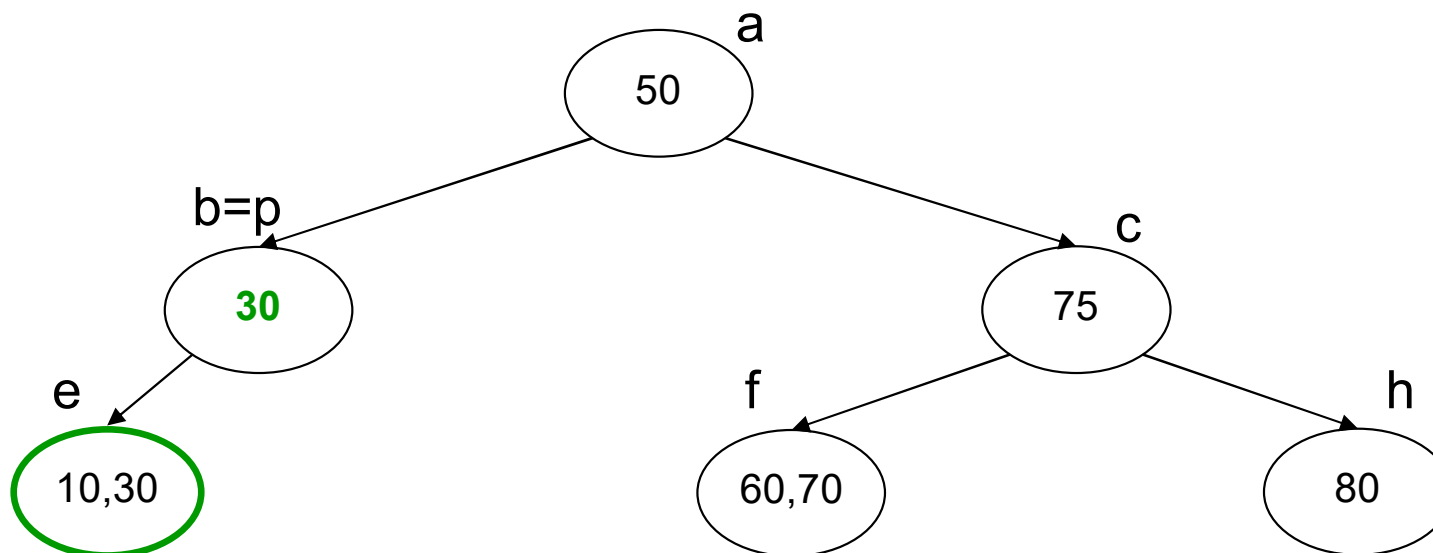
- Nesse caso, as chaves do nó **d** são combinadas com aquelas restantes do nó **e** e a chave 30 do nó pai **b** para obter o novo nó **e**, com os valores 10 e 30
 - Note que **d** ainda reside em disco mas não é mais utilizado



Remoção $X = 40$

R	W
4	1

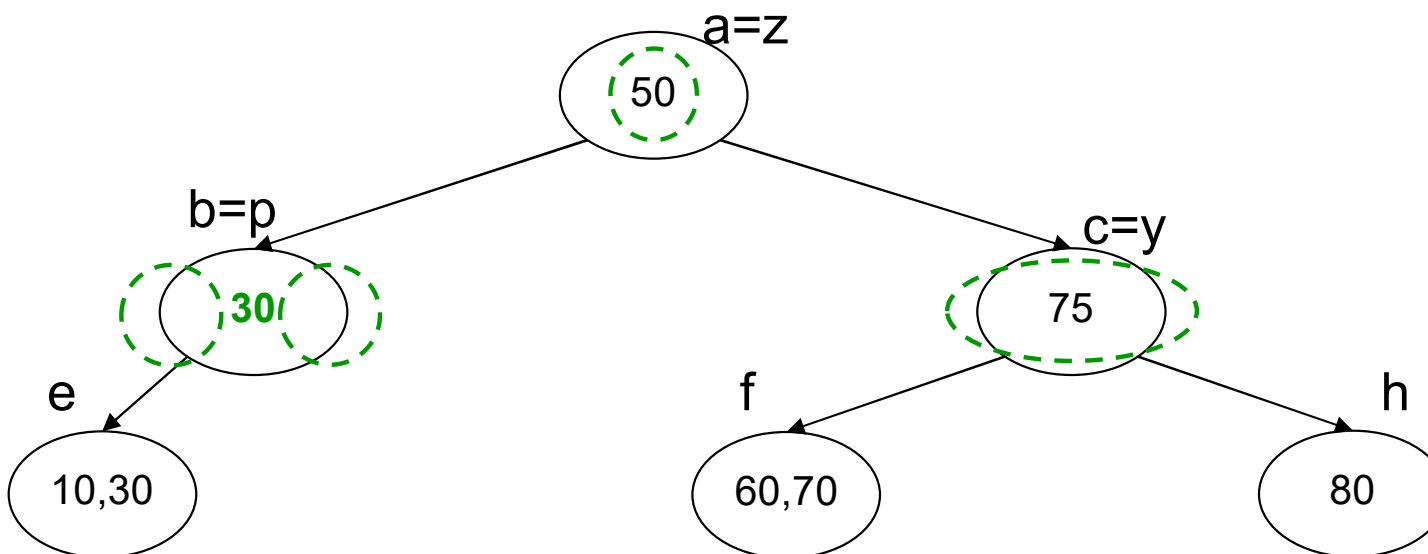
- Nesse caso, as chaves do nó **d** são combinadas com aquelas restantes do nó **e** e a chave 30 do nó pai **b** para obter o novo nó **e**, com os valores 10 e 30
 - Note que **d** ainda reside em disco mas não é mais utilizado
- Isso, porém, deixa **b** com uma chave a menos do permitido



Remoção $X = 40$

R	W
5	1

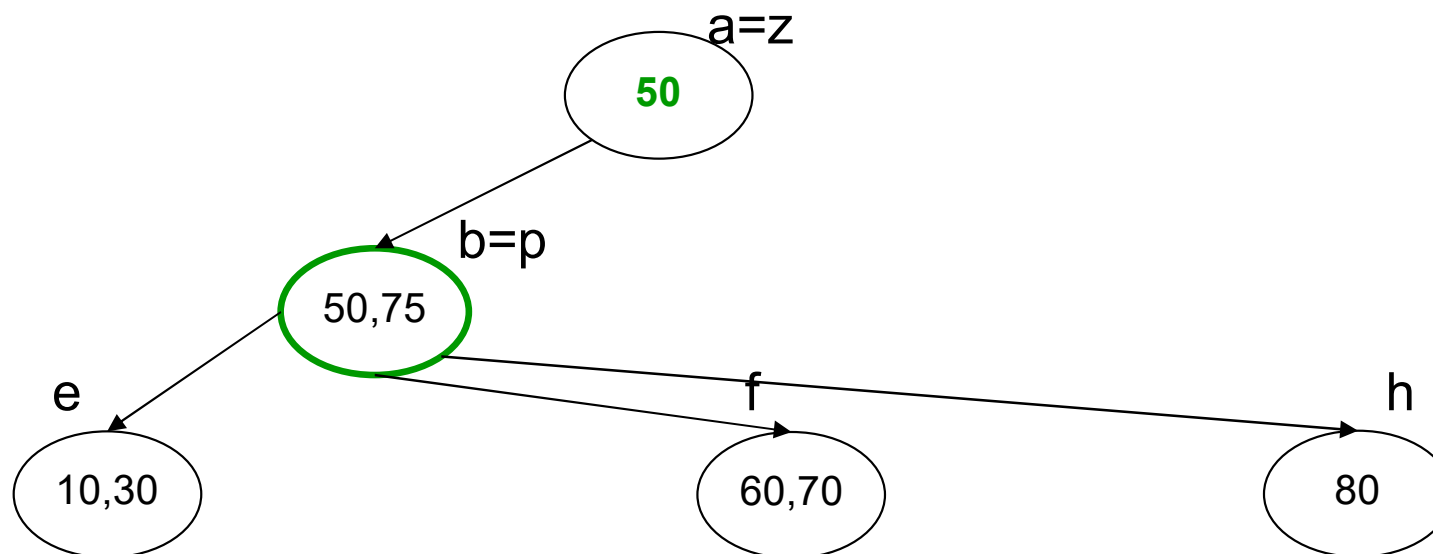
- ❑ Isso, porém, deixa **b** com uma chave a menos do permitido
- ❑ O seu irmão **c** não tem chaves extras para emprestar e, por isso, as chaves restantes de **b** são combinadas, com o valor 50 do pai **a** e $\lceil m/2 \rceil - 1$ chaves de **c**, para se obter o novo nó **b**



Remoção $X = 40$

R	W
5	2

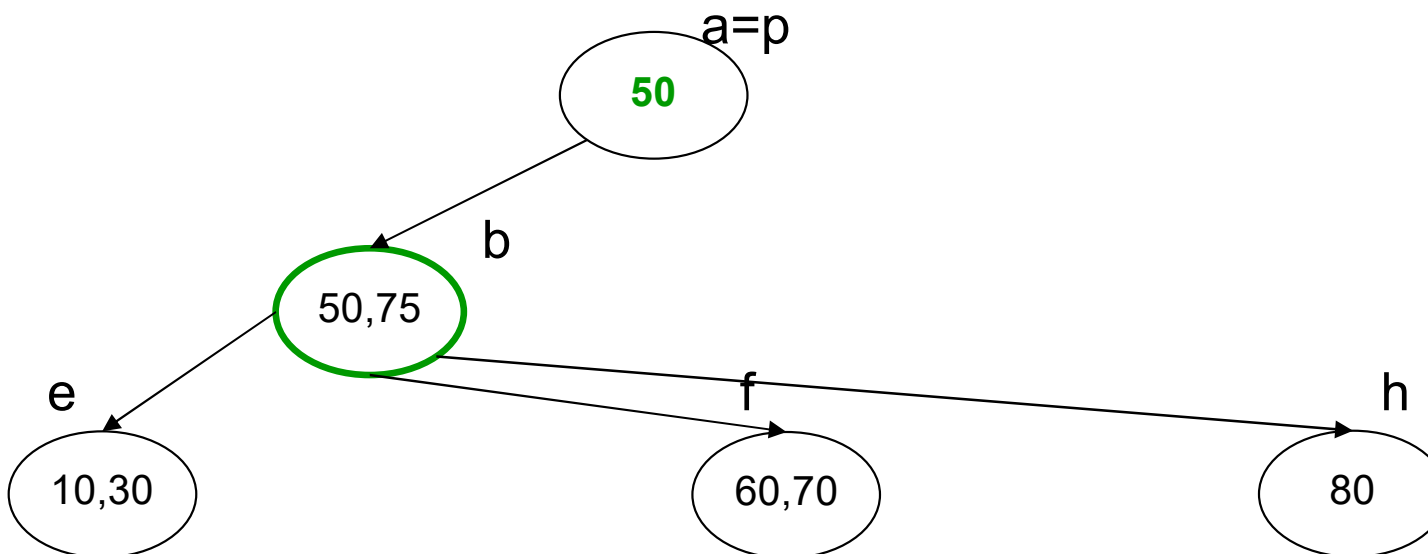
- ❑ Isso, porém, deixa **b** com uma chave a menos do permitido
- ❑ O seu irmão **c** não tem chaves extras para emprestar e, por isso, as chaves restantes de **b** são combinadas, com o valor 50 do pai **a** e $\lceil m/2 \rceil - 1$ chaves de **c**, para se obter o novo nó **b**
 - Note que **c** ainda reside em disco mas não é mais utilizado



Remoção $X = 40$

R	W
5	2

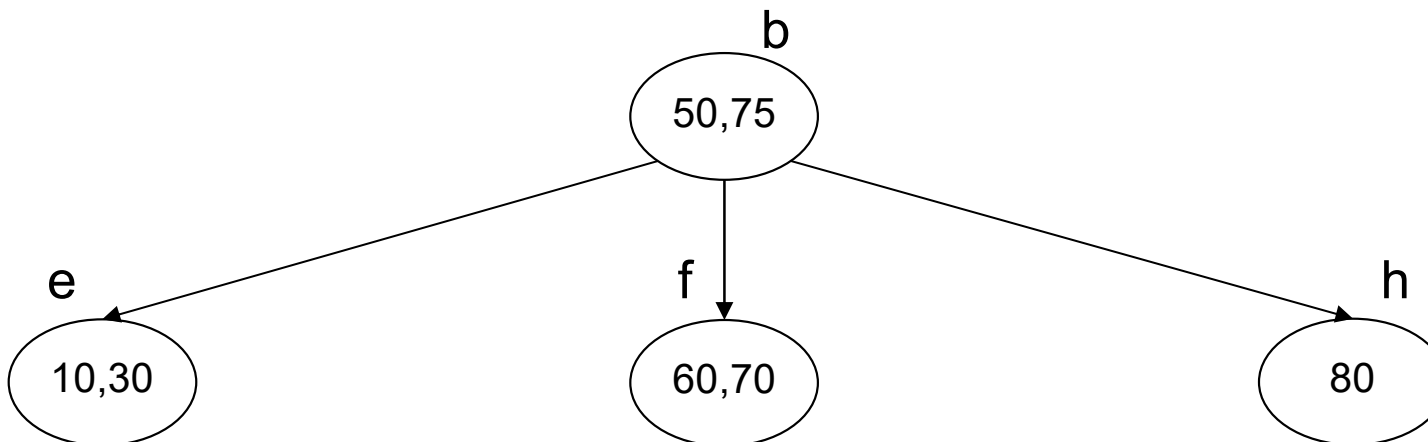
- ❑ Isso, porém, deixa **b** com uma chave a menos do permitido
- ❑ O seu irmão **c** não tem chaves extras para emprestar e, por isso, as chaves restantes de **b** são combinadas, com o valor 50 do pai **a** e $\lceil m/2 \rceil - 1$ chaves de **c**, para se obter o novo nó **b**
 - Note que **c** ainda reside em disco mas não é mais utilizado
- ❑ O nó raiz **a** ficou vazio e é descartado
 - $T \leftarrow A_0$



Remoção $X = 40$

R	W
5	2

- Isto toma um total de quatro acessos adicionais (1 para buscar cada um dos nós **d** e **c**, 1 para regravar cada um dos nós **e** e **b** alterados), totalizando 7 acessos

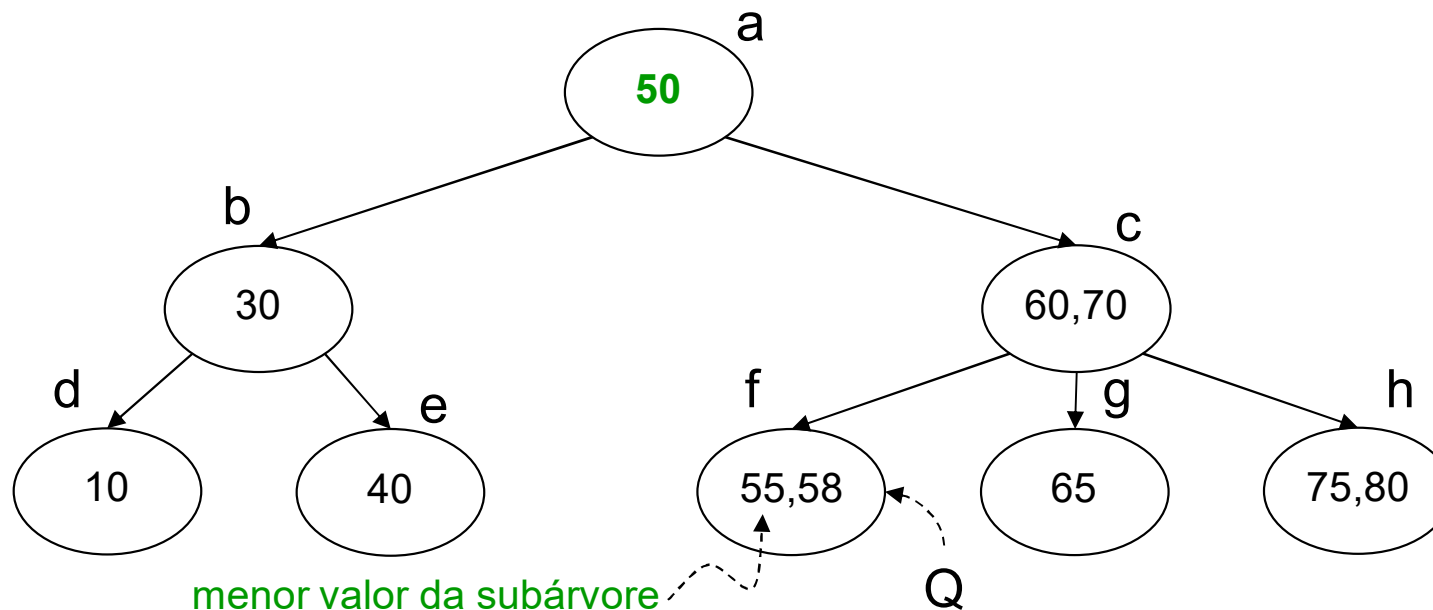


Remoção

- ❑ Quando a chave X a ser eliminada não se encontra em uma folha então pode ser feita uma simples transformação que permitirá efetuar a remoção como no caso de um nó folha
- ❑ Suponha que $K_i = X$ no nó P que tem a forma
 - $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$ com $1 \leq i \leq n$
- ❑ Como P não é uma folha então $A_i \neq 0$
- ❑ Podemos determinar Y como o menor valor de chave na subárvore A_i , que se encontrará em um nó folha Q
- ❑ Substitua K_i por Y no P e grave o novo P
- ❑ Isso nos deixa com o problema de eliminar Y do Q
- ❑ Note que estão sendo mantidas as propriedades de pesquisa da árvore

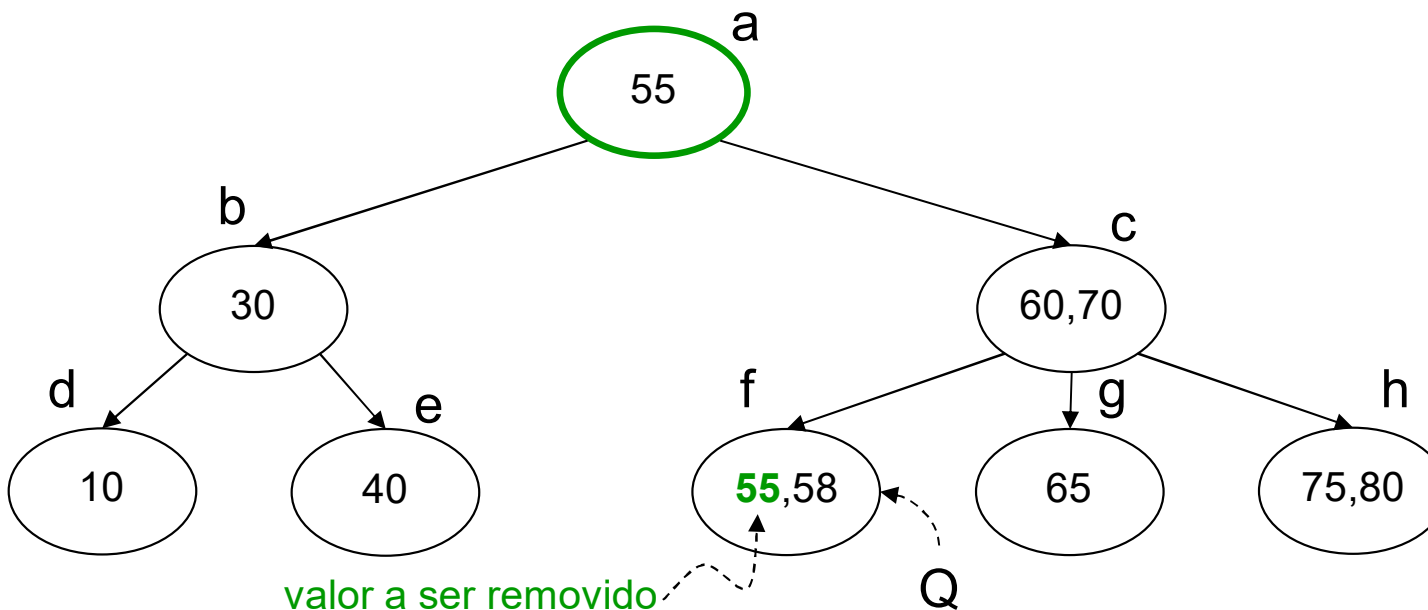
Remoção

- Por exemplo, a remoção de $X = 50$ pode ser conseguida substituindo primeiro 50 do nó **a** por 55 do nó **f** e então procedendo com a remoção de 55 do nó **f**



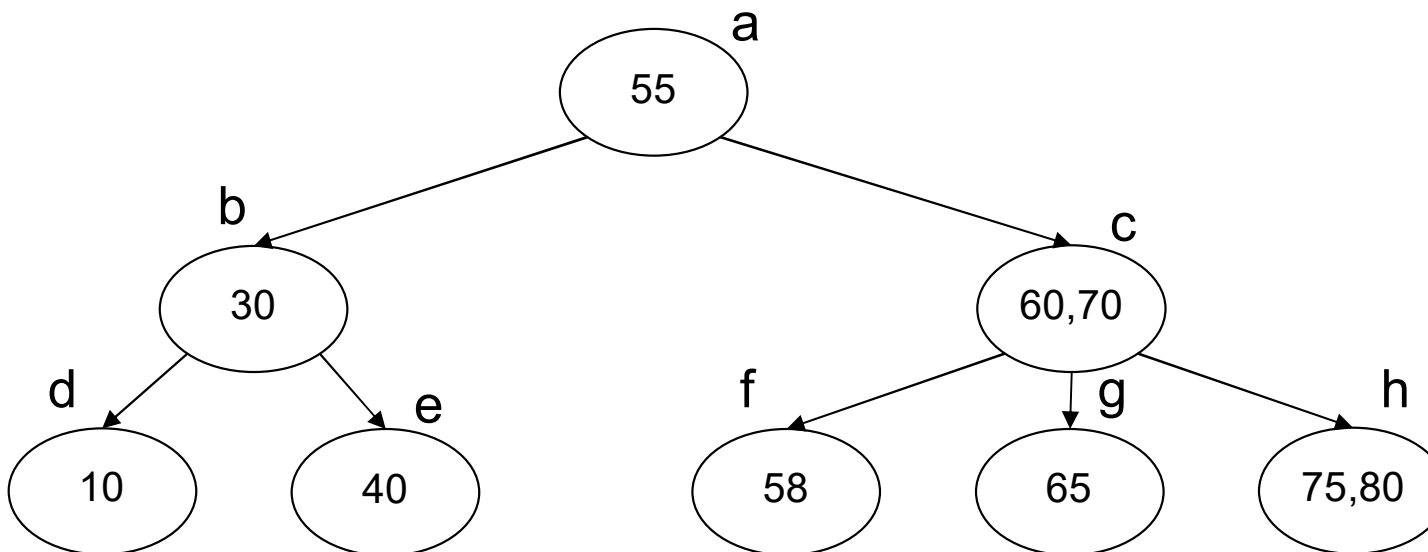
Remoção

- Por exemplo, a remoção de $X = 50$ pode ser conseguida substituindo primeiro 50 do nó **a** por 55 do nó **f** e então procedendo com a remoção de 55 do nó **f**



Exercício

- ❑ Remova a chave 55 da árvore B, anotando quantos acessos a disco são necessários



Algoritmo de Remoção

- ❑ No algoritmo de remoção, **deleteB**, assume-se que a chave X deve ser removida da árvore T tipo B da ordem **m** que reside em disco

Algoritmo de Remoção

```
procedure deleteB(T, X)
  1 (p,i,achou) ← mSearch(T,X);
  2 if not achou then return; endif // X não está em T
  3 seja p da forma:  $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$  com  $K_i = X$ 
  4 if  $A_0 \neq 0$  then
  5   // remoção a partir de não-folha, encontre chave p/mover
  6    $q \leftarrow A_i$ ; // ir para subárvore direita
  7   seja q da forma:  $n_q, A_{q_0}, (K_{q_1}, A_{q_1}), \dots, (K_{q_{n_q}}, A_{q_{n_q}})$ 
  8   while  $A_{q_0} \neq 0$  do // q não é uma folha
  9      $q \leftarrow A_{q_0}$ ; // procurar uma folha
  10  endwhile
  11  substitua  $K_i$  por  $K_{q_1}$  no nó p e grave em disco o nó p alterado
  12   $p \leftarrow q$ ;  $i \leftarrow 1$ ;
  13  seja  $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$  definido pelo novo p
  endif
  // suprima  $K_i$  do nó p, uma folha
  14  $p = n, A_0, (K_1, A_1), \dots, (K_n, A_n)$  remova  $(K_i, A_i)$  de p e substitua n por n-1;
```

Algoritmo de Remoção

```
15 while  $n < \lceil m/2 \rceil - 1$  and  $p \neq T$  do
16   if  $p$  tem um irmão direito  $y$  mais próximo then
17     seja  $y$ :  $ny, Ay_0, (Ky_1, Ay_1), \dots, (Ky_{ny}, Ay_{ny})$ ;
18     seja  $z$ :  $nz, Az_0, (Kz_1, Az_1), \dots, (Kz_{nz}, Az_{nz})$  o pai de  $p$  e  $y$ 
19     seja  $j$  tal que  $p = Az_{j-1}$  e  $y = Az_j$ 
20     if  $ny \geq \lceil m/2 \rceil$  then //  $y$  pode emprestar, redistribua as chaves
21       atualize  $p$ :  $(K_{n+1}, A_{n+1}) \leftarrow (Kz_j, Ay_0)$ ;  $n \leftarrow n+1$ ;
22       atualize  $z$ :  $Kz_j \leftarrow Ky_1$ ;
23       atualize  $y$ :  $(ny, Ay_0, (Ky_1, Ay_1), \dots) \leftarrow (ny-1, Ay_1, (Ky_2, Ay_2), \dots)$ ;
24       Escreva nós  $p, z, y$  para disco
25       return;
    endif
    // combinar  $p, Kz_j$  e  $y$ 
26    $r \leftarrow 2 * \lceil m/2 \rceil - 2$ ;
27   Escreva  $r, A_0, (K_1, A_1), \dots, (K_n, A_n), (Kz_j, Ay_0), (Ky_1, Ay_1), \dots, (Ky_{ny}, Ay_{ny})$ 
    como novo nó  $p$  para disco
28    $(nz, Az_0, \dots) \leftarrow (nz-1, Az_0, \dots, (Kz_{j-1}, Az_{j-1}), (Kz_{j+1}, Az_{j+1}), \dots)$ ;
29    $p \leftarrow z$ ; // continuar processo no nó pai
  else
30    //  $p$  deve ter um irmão esquerdo, o que é simétrico às linhas 17-29
  endif
31 endwhile
```

Algoritmo de Remoção

```
32   if  $n \neq 0$  then
33       Escreva nó  $p:n, A_0, (K_1, A_1) \dots, (K_n, A_n)$  para disco;
34   else
35        $T \leftarrow A_0$ ; // nova raiz
36   endif
end deleteB
```

Algoritmo de Remoção

- ❑ Nas linhas 4-13 é feito o tratamento para o caso de uma remoção em um nó não folha, procurando um elemento para trocar e proceder a remoção em uma folha
- ❑ Nas linhas 21-25 são efetuadas as trocas de chaves (“empréstimo” de chaves)
- ❑ Nas linhas 26-28 é efetuada a reunião de chaves
- ❑ As linhas 32-36 somente são executadas se o processo chegou até a raiz

Análise deleteB

- ❑ A busca de X (linha 1), junto com a pesquisa da folha Q nas linhas 5-10 precisam de h acessos, sendo h o número de níveis em T
- ❑ No caso que p não é um nó folha (linha 3) então ele é modificado e gravado conforme a linha 11
- ❑ Assim, a quantidade máxima de acessos nas linhas 1-13 é $h+1$
- ❑ Começando de um nó folha, cada iteração do laço **while** das linhas 15-31, move p um nível para cima na árvore
 - Por isso, podem ser feitas no máximo $h-1$ iterações desse laço
 - Admitindo-se, que todos os nós desde a raiz da T até a folha p , encontram-se na memória principal, os únicos acessos adicionais necessários são para os nós irmãos e para regravar os nós que foram alterados
 - O pior caso acontece quando são feitas $h-1$ iterações e a última resulta em terminação na linha 25 (ou local correspondente da linha 30)
 - Portanto, a quantidade máxima de acessos para esse laço é $(h-1)$ para os irmãos, mais $(h-2)$ atualizações na linha 27 (ou o local correspondente na linha 30), mais as 3 atualizações da linha 24 (ou 30) para término, totalizando, $(h-1)+(h-2)+3 = 2*h$
- ❑ Portanto, a quantidade máxima de acessos necessários para uma remoção é de $(h+1)+(2*h) = 3*h+1$

Análise deleteB

- ❑ O tempo da remoção pode ser reduzido em troca de dispêndio de espaço no disco e um ligeiro aumento do tamanho do nó, incluindo um bit de remoção F_i , para cada valor de chave K_i em um nó
- ❑ Com isso, podemos posicionar $F_i=1$ se K_i não foi suprimido e $F_i=0$ quando foi
- ❑ Portanto, não ocorre uma remoção física e este tipo de remoção exige no máximo $h+1$ acessos (h para localizar o nó contendo X e 1 para gravar esse nó, com bit de remoção posicionado em 0)
- ❑ Com esta estratégia, a quantidade de nós da árvore nunca decresce, porém o espaço usado pelas entradas removidas pode ser usado novamente, durante novas inserções
- ❑ Como resultado, a estratégia teria pouca influência sobre os tempos de busca e inserção (quando m é grande, o número de níveis aumenta muito devagar com o aumento do número de chaves)
- ❑ O tempo de inserção pode até mesmo diminuir ligeiramente, por causa da habilidade para reusar o espaço de chaves removidas, o que pode evitar a necessidade de fracionar nós

Resumo

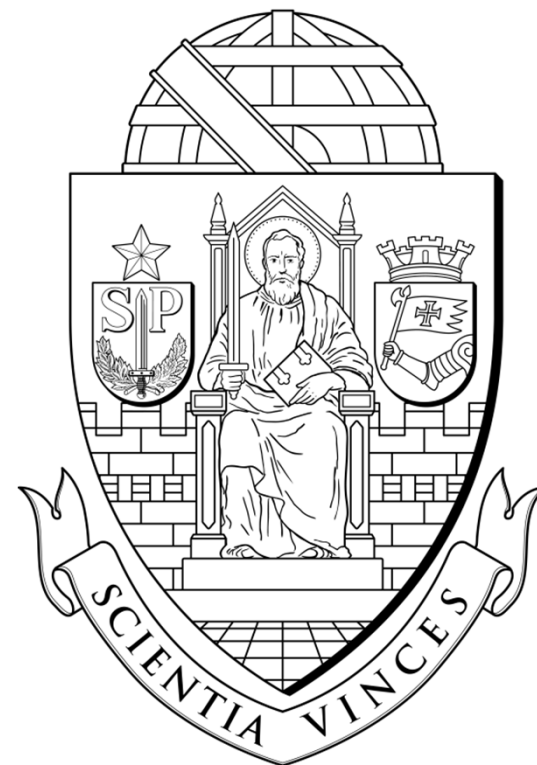
- ❑ Árvores B de grau m são simples, de fácil manutenção e eficientes
- ❑ O custo computacional para buscar, inserir e remover é proporcional à altura da árvore $O(h) = O(\log_m N)$, onde N é o número de elementos (chaves) na árvore
- ❑ O espaço utilizado pelos dados é, no mínimo, 50% do espaço reservado
- ❑ As árvores B crescem e diminuem automaticamente e nunca requerem uma reorganização completa

Copyright© Apresentação 2026 por
José Augusto Baranauskas
Universidade de São Paulo

Professores são convidados a utilizarem esta apresentação da maneira que lhes for conveniente para uso não comercial, desde que esta nota de *copyright* permaneça intacta.

Slides baseados em:

- ❑ Horowitz, E. & Sahni, S.; Fundamentos de Estruturas de Dados, Editora Campus, 1984.
- ❑ Wirth, N.; Algoritmos e Estruturas de Dados, Prentice/Hall do Brasil, 1989.



USP